

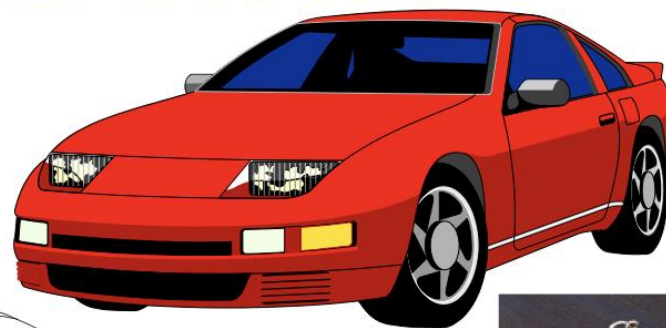
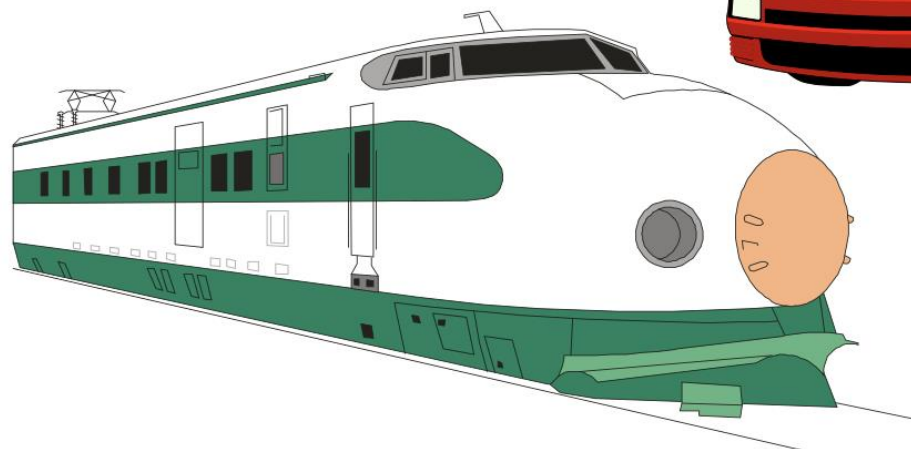
Introduction to embedded systems and IoT

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen



Embedded System = *Computers Inside a Product*



Philip Koopman

Emeritus Professor, Electrical and
Computer Engineering

Course overview

	Monday	Tuesday	Wednesday	Thursday	Friday		Week	Topic of Each Week	Homework
Jan	5	6	7	8	9		1	Introduction to embedded systems and IoT	
	12	13	14	15	16		2	Electronics and digital logic	
	19	20	21	22	23		3	C programming basics	
	26	27	28	29	30		4	Embedded software organization	
Feb	2	3	4	5	6		5	Design and development process	Feb 4: HW1
	9	10	11	12	13				
	16	17	18	19	20				
	23	24	25	26	27				
Mar	2	3	4	5	6		6	I/O ports, switch and LED interfacing	
	9	10	11	12	13		7	Digital sampling, ADC and DAC	
	16	17	18	19	20		8	Memory and processor bus, timer and counters	
	23	24	25	26	27		9	Timing constraints and real-time CPU sched	Mar 25: HW2
Apr	30	31	1	2	3		10	Internet of things basics	
	6	7	8	9	10		11	Bluetooth and CAN	
	13	14	15	16	17		12	Machine learning introduction	
	20	21	22	23	24		13	Embedded machine learning applications	Apr 22: HW3
	27	28	29	30	1		14	Review for final exam and guest lectures	
May	4	5	6	7	8				May 9: Final Exam 1pm to 3pm

Grading

- 60 points homework (3 homework, 20 points each)
- 40 points final exam
- 10 bonus points (capped at 10) for classroom interactions
 - Virtual lab session interaction

Mixture of lecture and virtual lab sessions

Virtual lab session

Lecture session



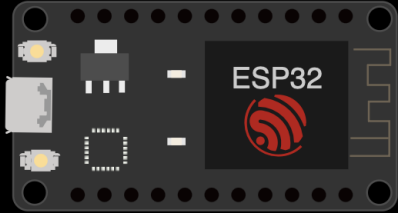
WOKWi
World's most advanced ESP32 simulator

[Discord Community](#) [LinkedIn Group](#) [Pricing](#)

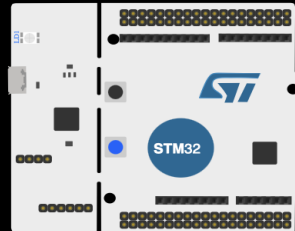
Simulate with Wokwi Online



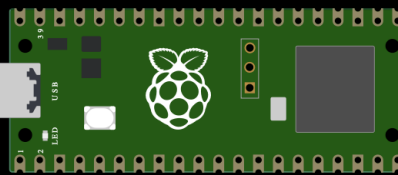
Arduino (Uno, Mega, Nano)



ESP32



STM32

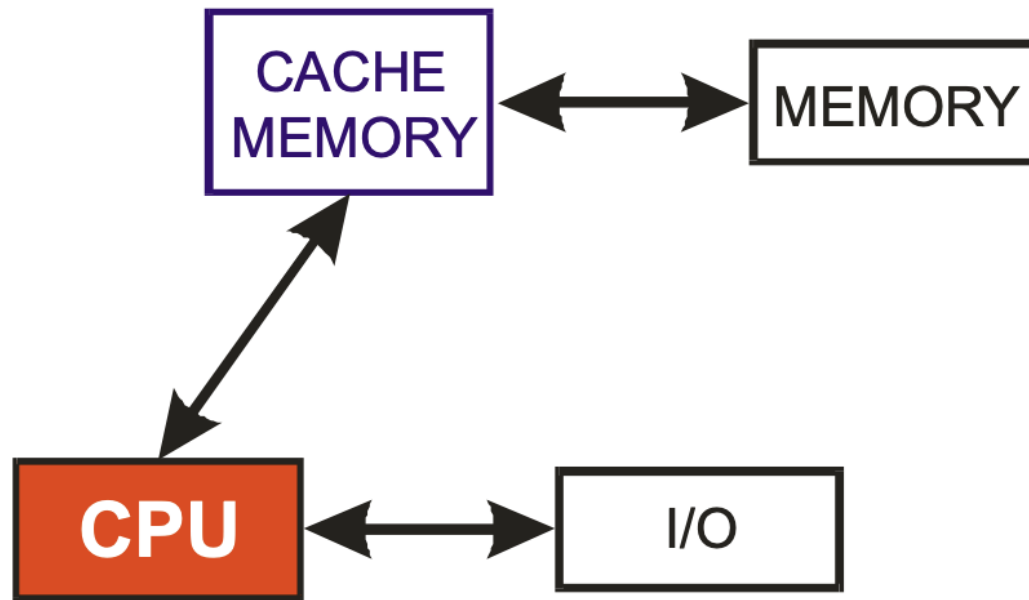


Pi Pico

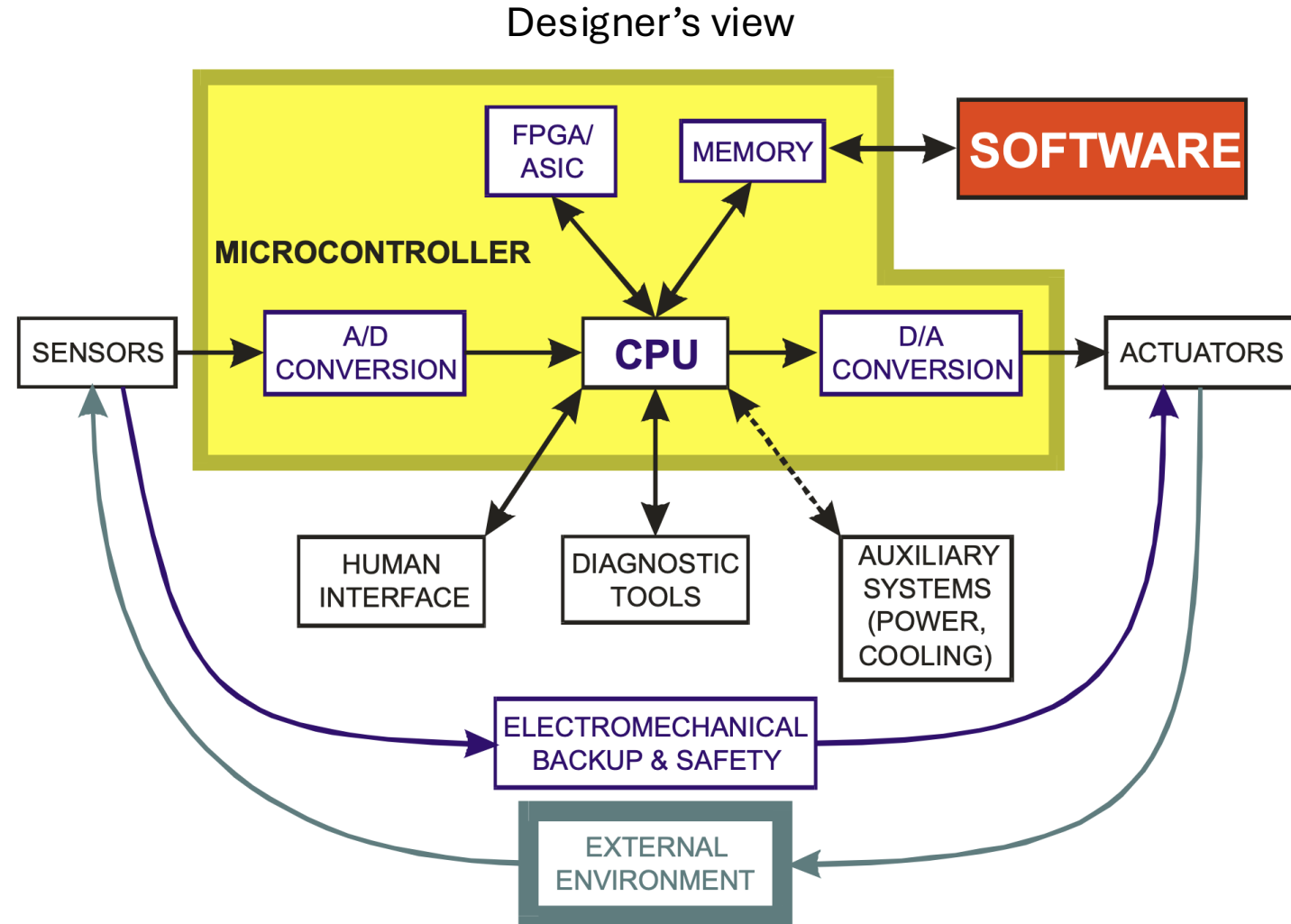
Instructor and teaching assistant

- Instructor: Yuning You
 - ynyou@cuhk.edu.cn
 - Chengdao 403
- Teaching assistants:
 - Yihuan Yao
 - 225015076@link.cuhk.edu.cn
- Office hour: By appointment

A common view of computers

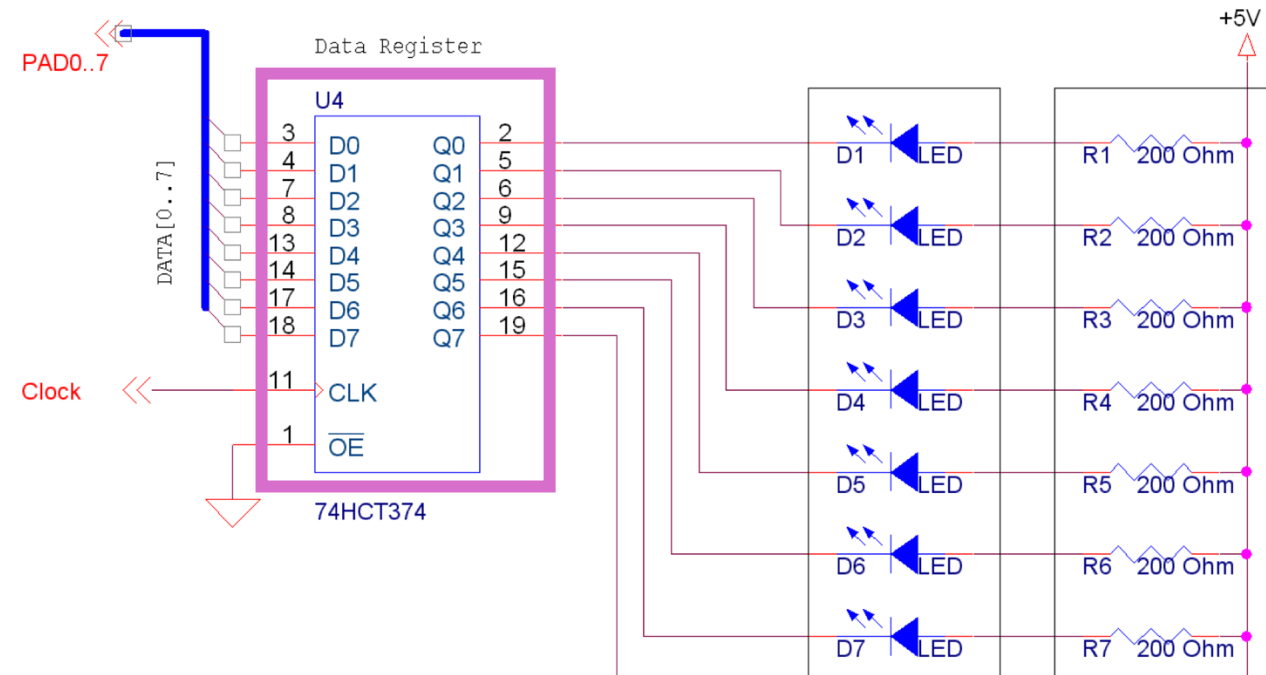
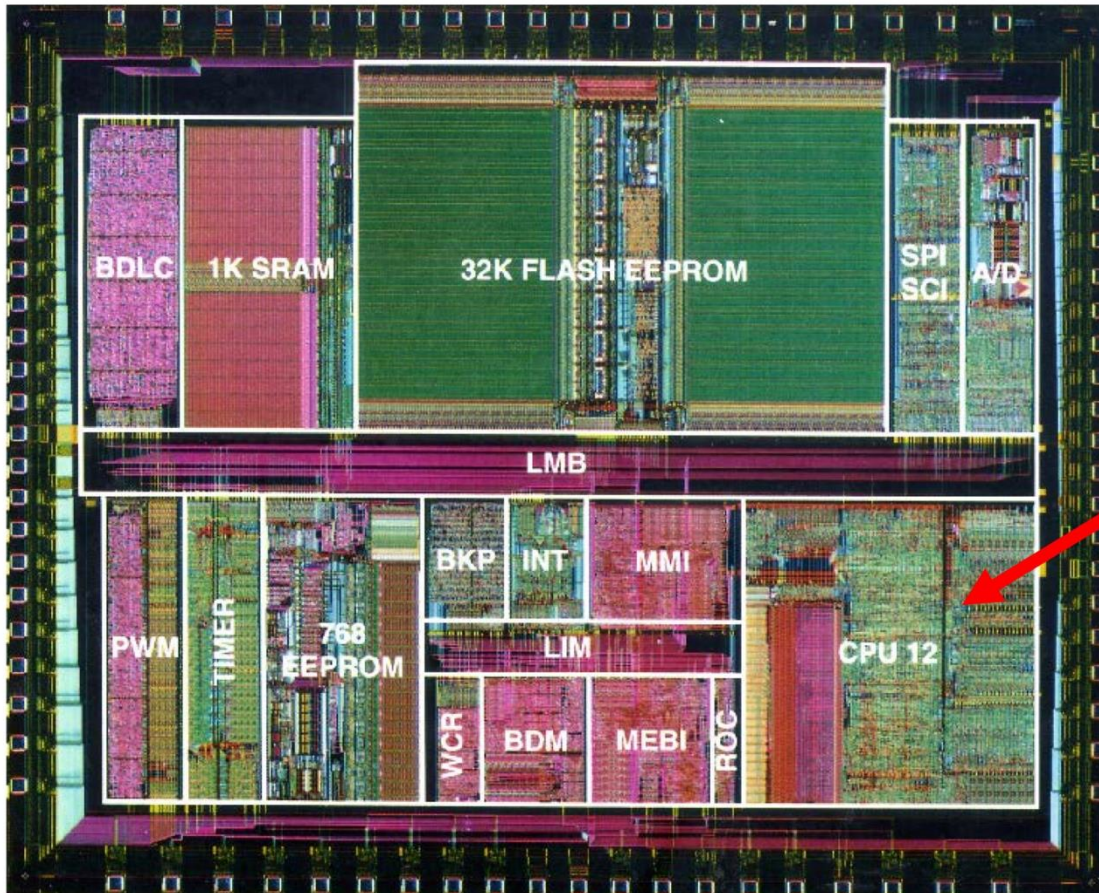


CPU: Central processing unit

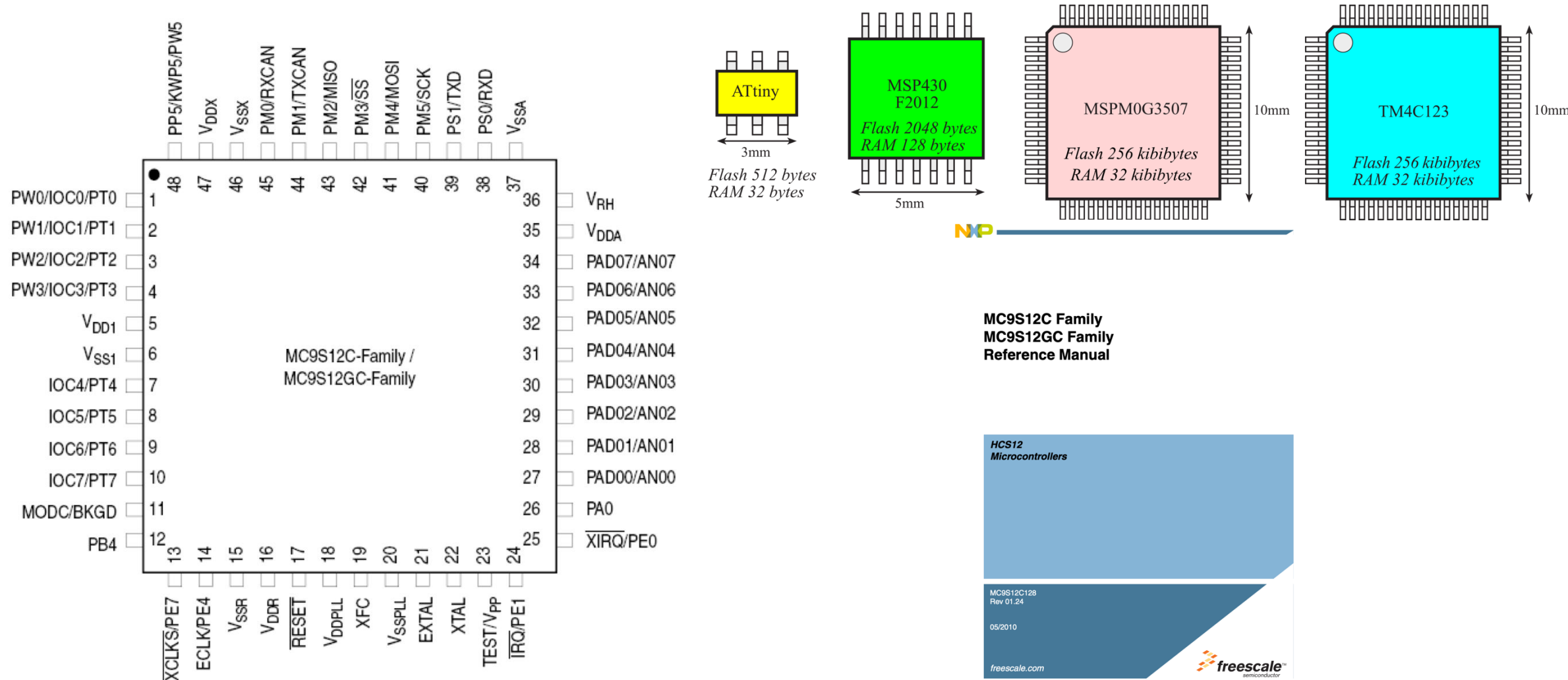


Microcontroller chip in practice

Chip diagram

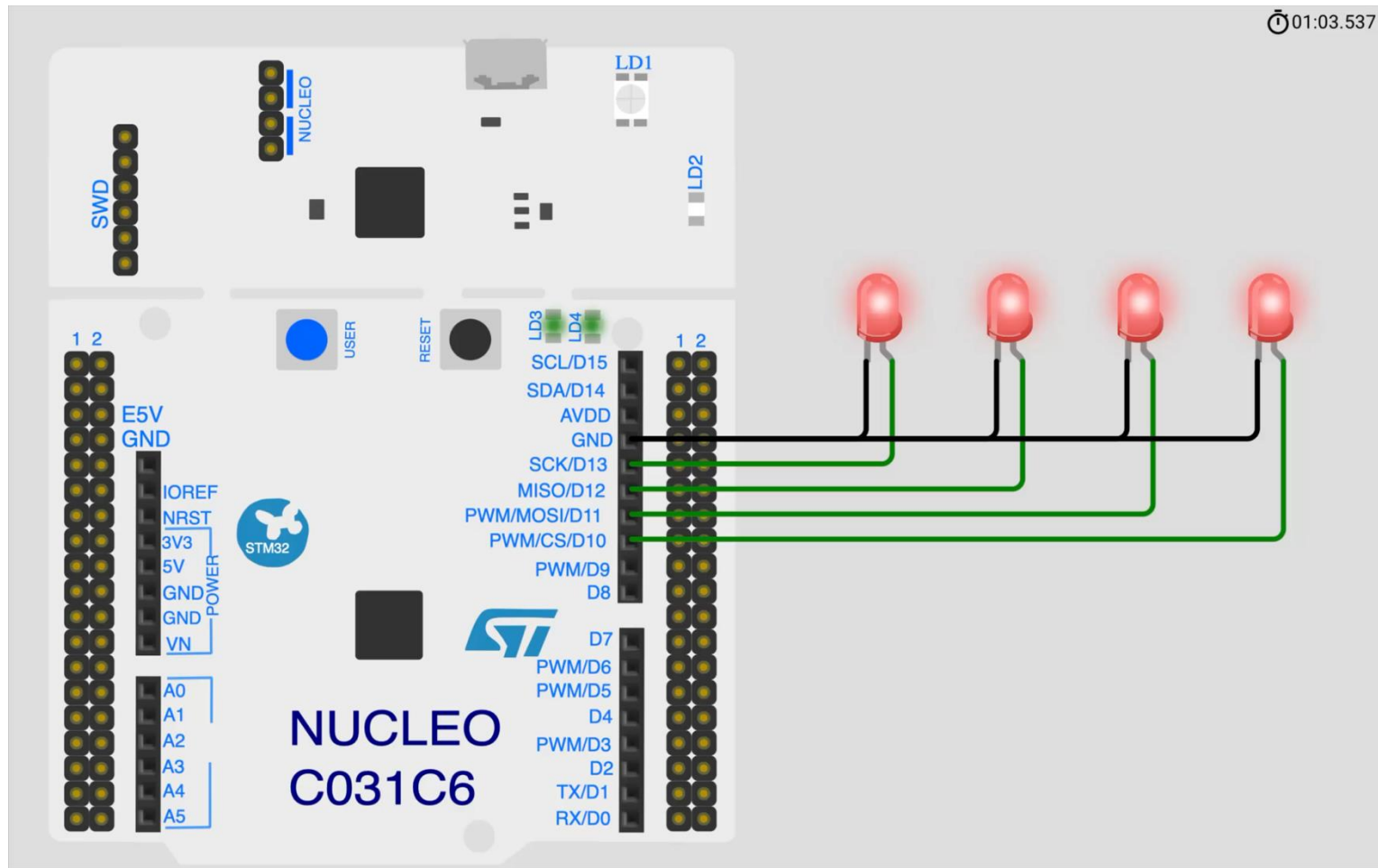


Actual chip diagrams are complicated



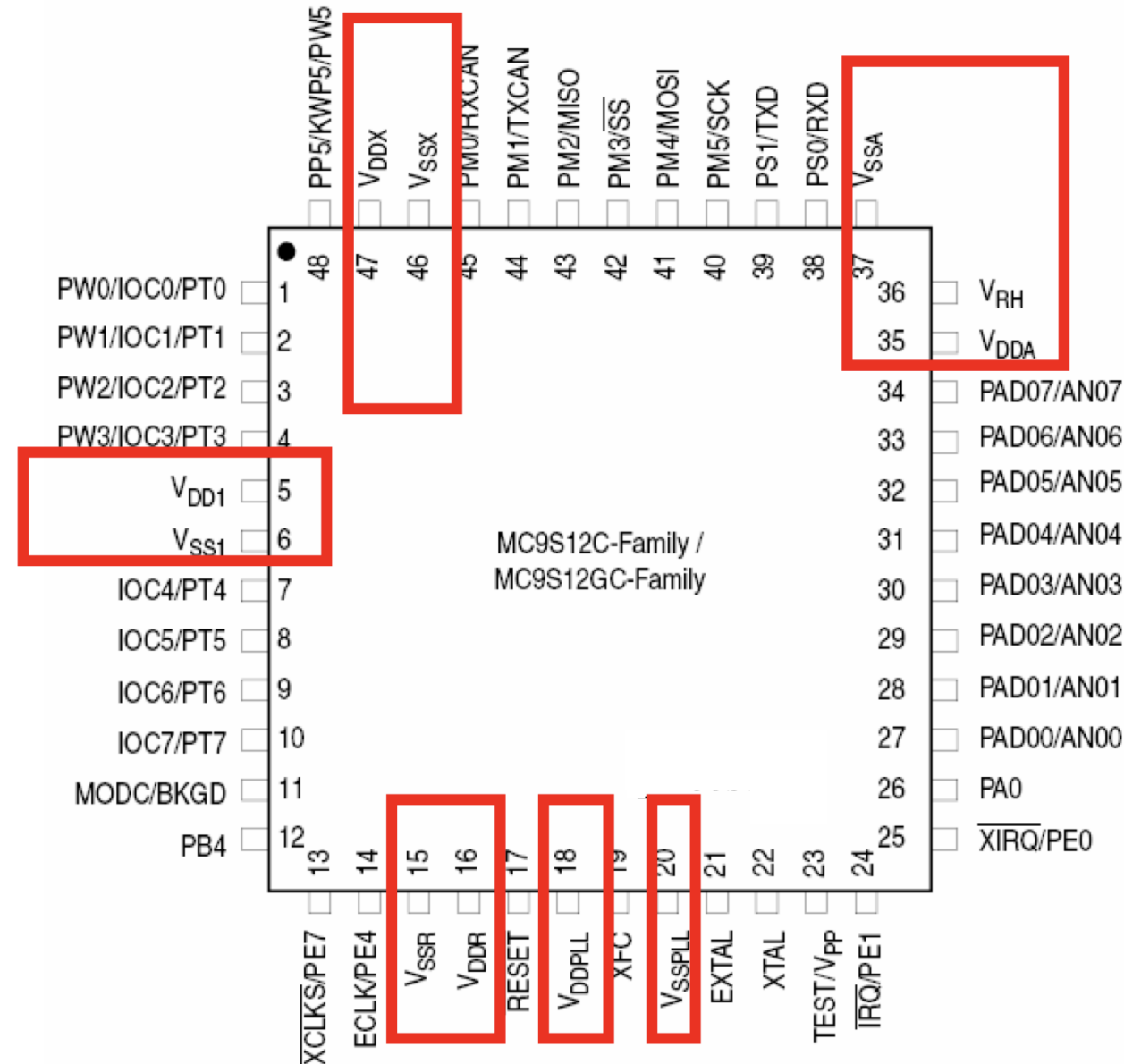
An example of running LEDs

- WOKWI – an online simulator: <https://wokwi.com/stm32>



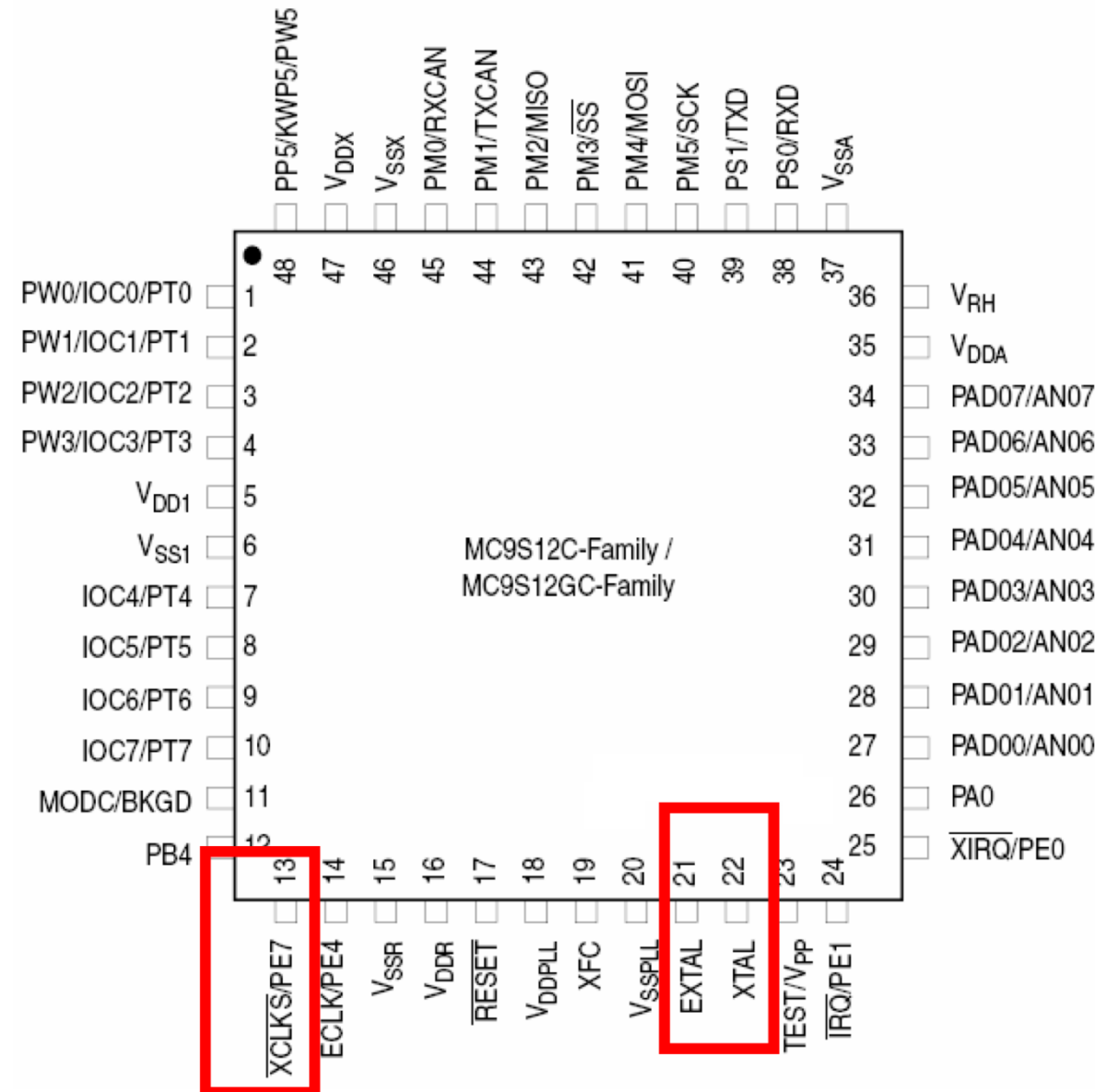
Power supply pins

- Pin names starts with “V”
- V_DD: Positive supply
- V_SS: Ground return
- V_DDX / V_S SX: Power supply pins for the domain X
- MC9S12C family
 - 5V for analog functions and 5V interface
 - 2.5V for internal logic



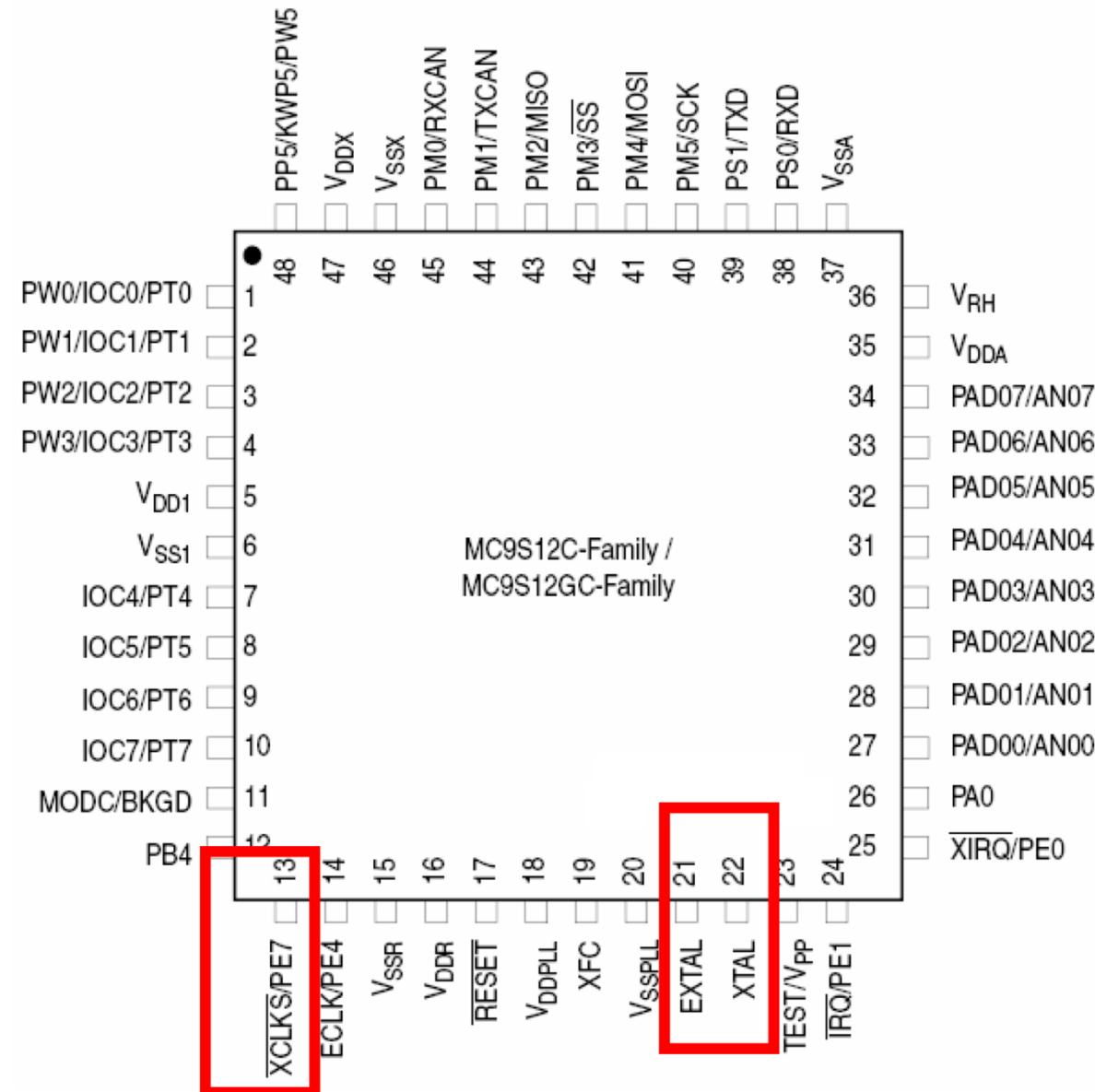
Oscillator/“clock” pins

- Periodic square wave for clocking the CPU
 - Internal low-quality oscillator
 - External high-quality oscillator
- XCLKS: External clock select
- EXTAL: External crystal in
- XTAL: Crystal Out



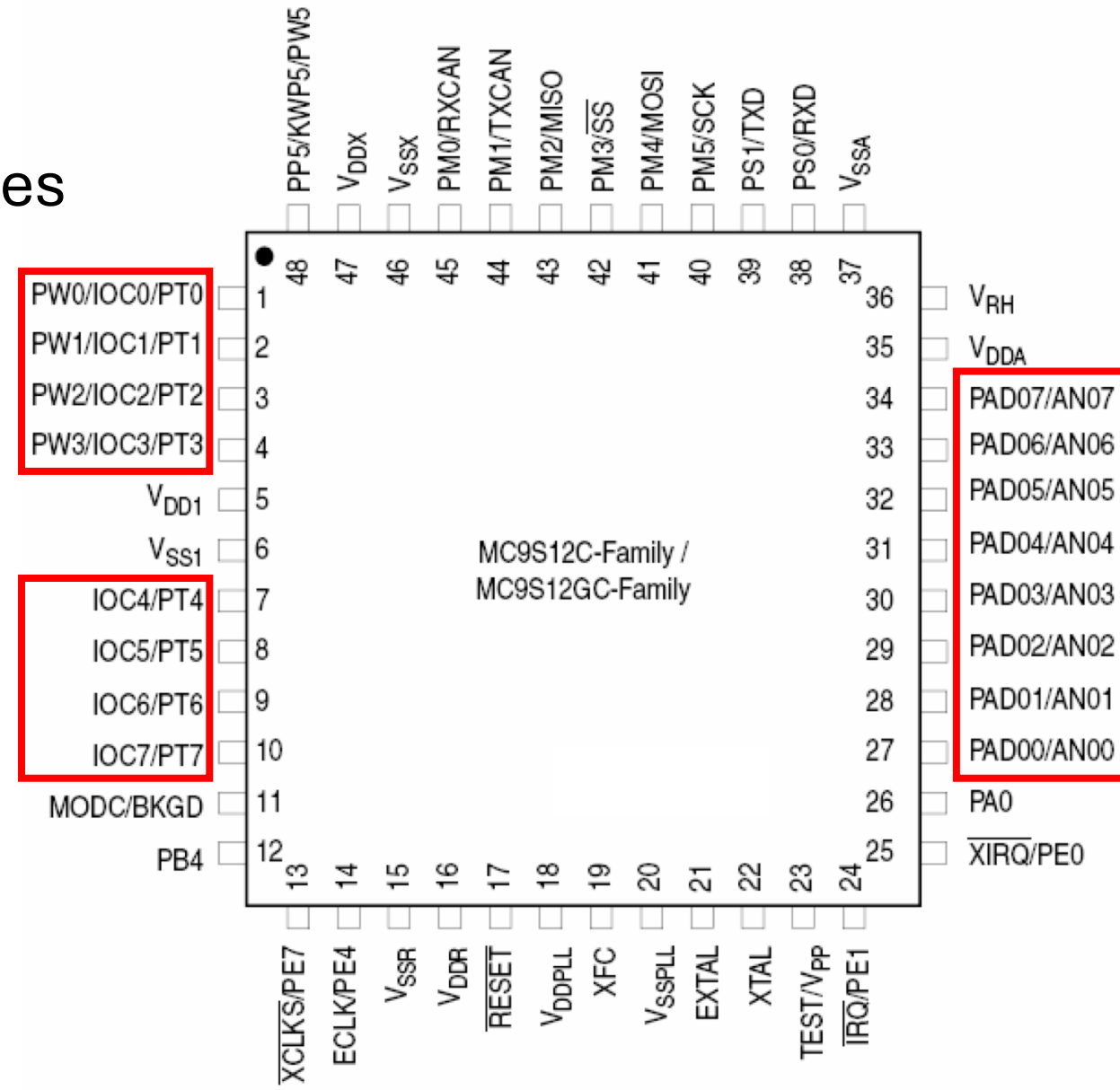
Oscillator/“clock” pins

- MC9S12C family oscillator speeds
 - 0.5 to 16 MHz on internal oscillator
 - 0.5 to 40 MHz external
- Why “slow” clock speeds?
 - Keeps costs down
 - Saves power
 - Fast enough for many purposes
 - Reduces emitted radio interference



I/O pins

- Pins are used for multiple purposes to reduce packaging costs
- PAD00..PAD07 // AN00 .. AN07 provides digital and analog I/O
- PT0..PT7 provides additional digital I/O
- PW0..PW3 provides hardware-assisted pulse generation



Registers for I/O pins

How do you get data on and off the pins?

- Interface to I/O is done via “registers” (a set of flip-flops on the chip)
- Write to registers to configure the pins – e.g., is it digital or analog?; in or out?
- Read/write other registers to actually do I/O
 - Read a byte from switches by reading register associated with digital inputs
 - Write a byte to LEDs by writing a register associated with digital outputs
 - But in both cases, first configure I/O via setting some register, then read/write values from a different register

Registers & memory maps for I/O pins

How do you access these registers?

- In some processors, an I/O instruction (x86: IN and OUT)
- In our processor, I/O is “memory mapped”
 - Use “load” and “store” instructions to special memory addresses

A memory map tells you where things are in memory

- Some of memory is RAM
- Some of memory is ROM
- Some of memory is I/O register space
- Look for the memory map in the data sheet

Random access memory (RAM)

- RAM: The workspace
 - RAM is your computer's short-term memory.
 - It's where the computer keeps the data it is actively using so that the CPU (the brain) can access it very quickly.
 - Volatile: RAM needs power to remember. The moment you turn off your computer, everything in RAM is erased. Your desk is cleared off
 - Read and Write: The CPU can both read data from and write data to RAM very quickly
 - Fast: RAM is much faster than storage drives (like SSDs or HDDs)
 - Temporary: It holds the operating system, the applications you have open (like your web browser, Word document), and the files you are currently working on.

Read only memory (ROM)

- ROM: The permanent instructions
 - ROM is your computer's permanent, non-volatile memory
 - Non-Volatile: The data is permanently written and remains even when the computer is turned off. The instruction manual is always in the drawer
 - Read-Only (Traditionally): As the name implies, data is typically "read" from ROM and not easily written to by the user. Its contents are "hard-coded" from the factory
 - Essential for Booting: It contains the firmware, the most basic instructions for the computer

von Neumann architecture

- Instructions are fetched from ROM on the same bus as data fetched from RAM
- Software is an ordered sequence of very specific instructions
- The processor executes the software by retrieving and interpreting these instructions one at a time

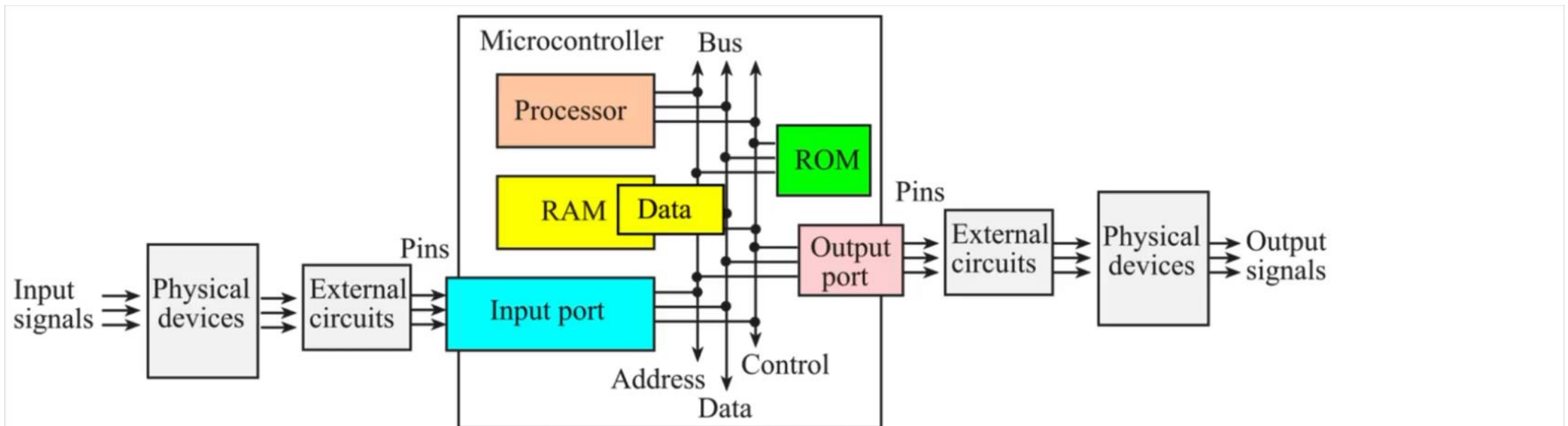


Figure 1.1.1. The basic components of a von Neumann computer including processor, memory and I/O connected with by a single bus.

Harvard architecture

- The processor can simultaneously fetch an instruction out of flash ROM using the ICode bus while it writes data into RAM using the system bus

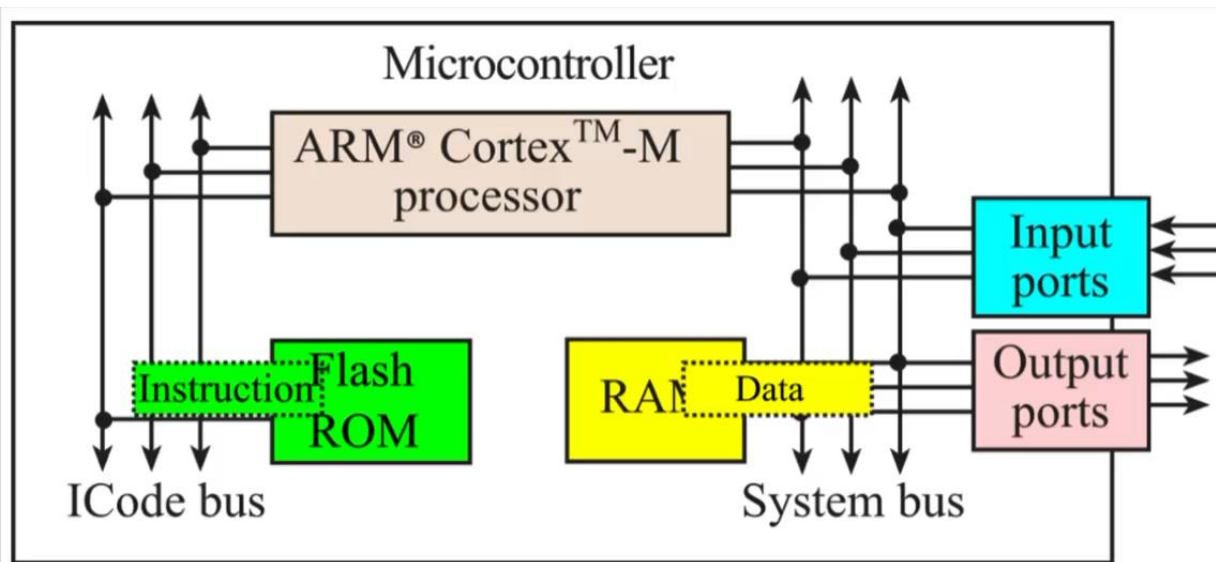
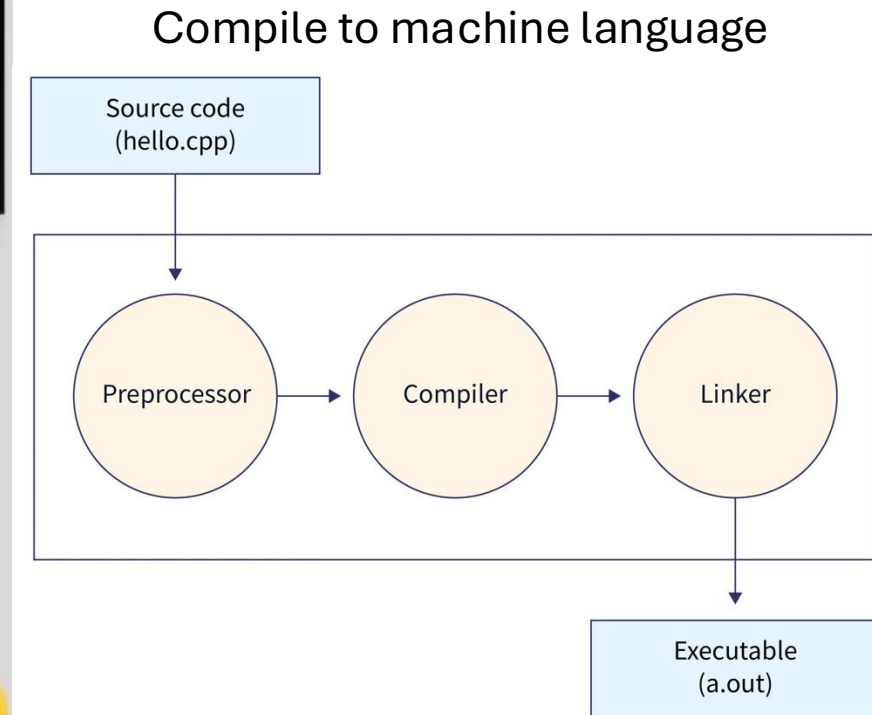
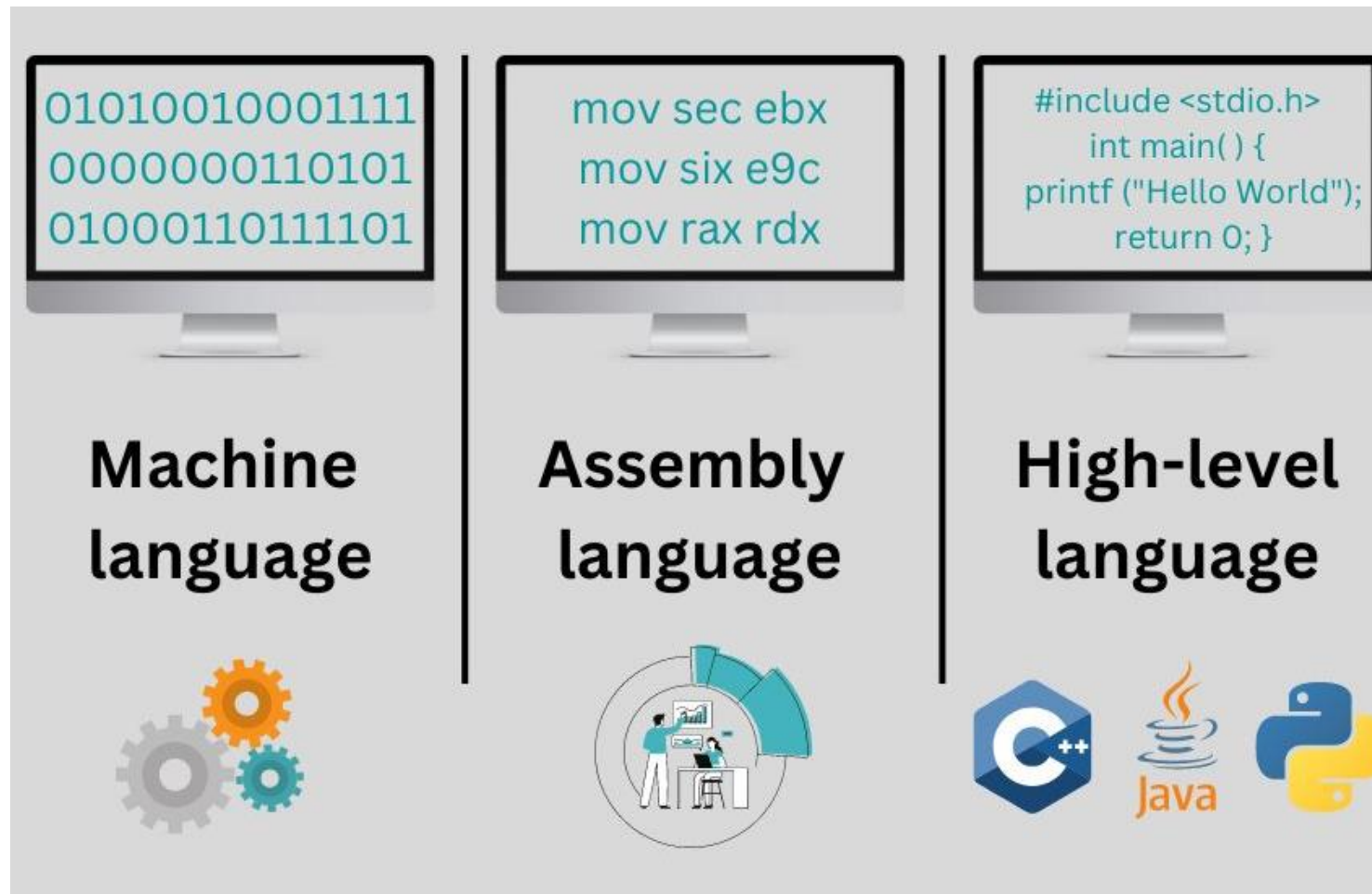


Figure 1.1.2. Harvard architecture of an ARM Cortex-M4. The three signals on each bus are address, data, control. If all these components exist on a single chip, it is called a microcontroller.

Programming languages

- The bridge between human and machines



C programming language

- The C programming language a foundational and efficient programming language
- It provides low-level control over system resources like memory
- Its power and simplicity have made it influential for decades.

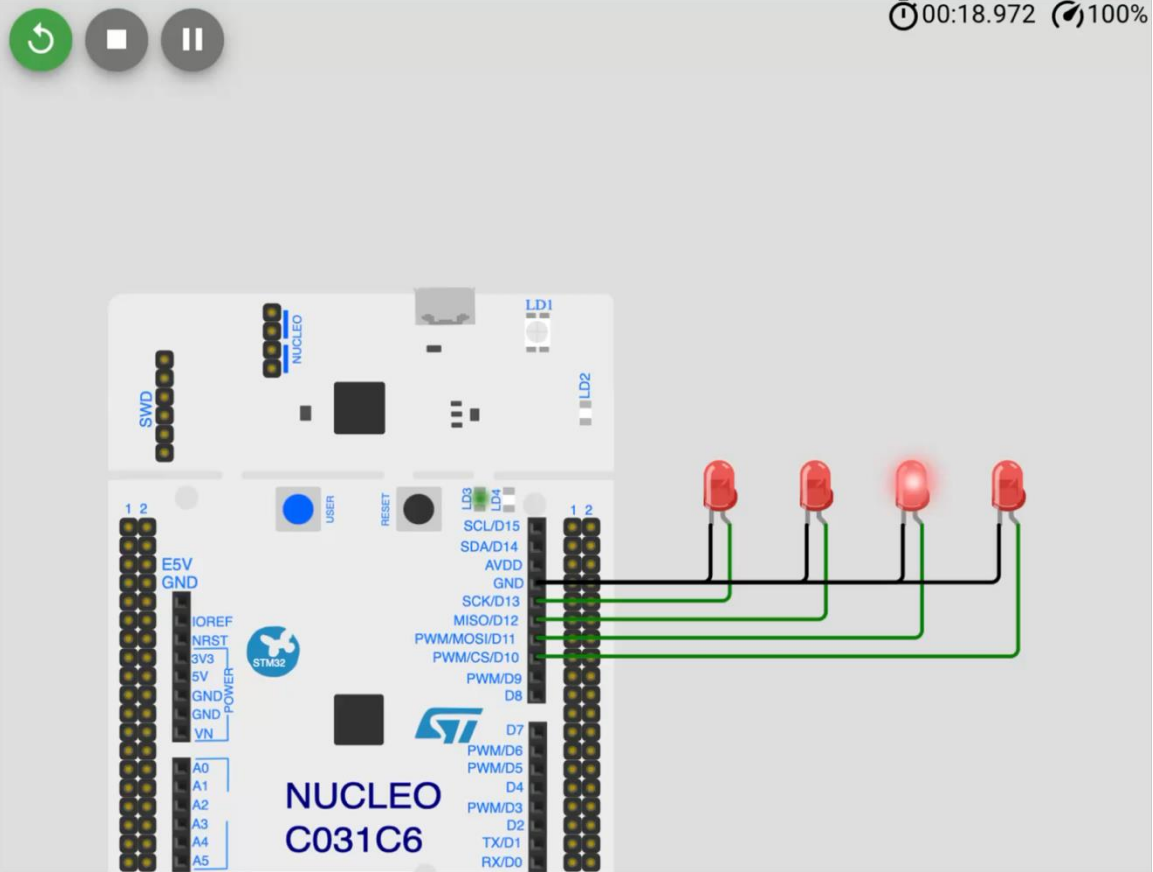
```
1 // 4-bit binary counter using LEDs (each LED toggles at different rates)
2
3 int ledPin[] = {13, 12, 11, 10};           // LED pins: MSB → LSB
4 int ledDelay[] = {8000, 4000, 2000, 1000}; // toggle interval (ms)
5 unsigned long lastChangeTime[4];          // store last toggle time of each LED
6 int ledState[4] = {LOW, LOW, LOW, LOW};   // store current state (LOW/HIGH)
7 unsigned long currTime;                   // current time variable
8
9 void setup() {
10     currTime = 0;                         // initialize timer
11     for (int i = 0; i < 4; i++) {
12         pinMode(ledPin[i], OUTPUT);       // set pin as output
13         digitalWrite(ledPin[i], ledState[i]); // turn LED off initially
14         lastChangeTime[i] = millis();     // record initial time
15     }
16 }
17
18 void loop() {
19     currTime = millis();                  // get current time
20
21     for (int i = 0; i < 4; i++) {         // check all LEDs
22         if (currTime - lastChangeTime[i] >= ledDelay[i]) {
23             // time to toggle the LED
24             ledState[i] = !ledState[i];   // flip LED state
25             digitalWrite(ledPin[i], ledState[i]); // apply new state
26             lastChangeTime[i] = currTime; // reset timer for that LED
27         }
28     }
29 }
```

Virtual lab session: Running LEDs

- A running LED demo:

<https://wokwi.com/projects/447390165530456065>

```
1 // 4-bit binary counter using LEDs (each LED toggles at different rates)
2
3 int ledPin[] = {13, 12, 11, 10};           // LED pins: MSB → LSB
4 int ledDelay[] = {8000, 4000, 2000, 1000}; // toggle interval (ms)
5 unsigned long lastChangeTime[4];           // store last toggle time of each LED
6 int ledState[4] = {LOW, LOW, LOW, LOW};    // store current state (LOW/HIGH)
7 unsigned long currTime;                     // current time variable
8
9 void setup() {
10     currTime = 0;                           // initialize timer
11     for (int i = 0; i < 4; i++) {
12         pinMode(ledPin[i], OUTPUT);          // set pin as output
13         digitalWrite(ledPin[i], ledState[i]); // turn LED off initially
14         lastChangeTime[i] = millis();        // record initial time
15     }
16 }
17
18 void loop() {
19     currTime = millis();                     // get current time
20
21     for (int i = 0; i < 4; i++) {           // check all LEDs
22         if (currTime - lastChangeTime[i] >= ledDelay[i]) {
23             // time to toggle the LED
24             ledState[i] = !ledState[i];      // flip LED state
25             digitalWrite(ledPin[i], ledState[i]); // apply new state
26             lastChangeTime[i] = currTime;    // reset timer for that LED
27         }
28     }
29 }
```



The screenshot displays a virtual lab session for a 4-bit binary counter using LEDs. On the left, the C++ code is shown, which initializes four LEDs (pins 13, 12, 11, 10) with different toggle intervals (8000, 4000, 2000, and 1000 ms). The code uses a loop to toggle the state of each LED based on the current time. On the right, the hardware simulation is shown, featuring a NUCLEO C031C6 board. Four red LEDs are connected to the board's pins, labeled LD1, LD2, LD3, and LD4. The simulation interface includes a timer showing 00:18.972 and a 100% zoom level.

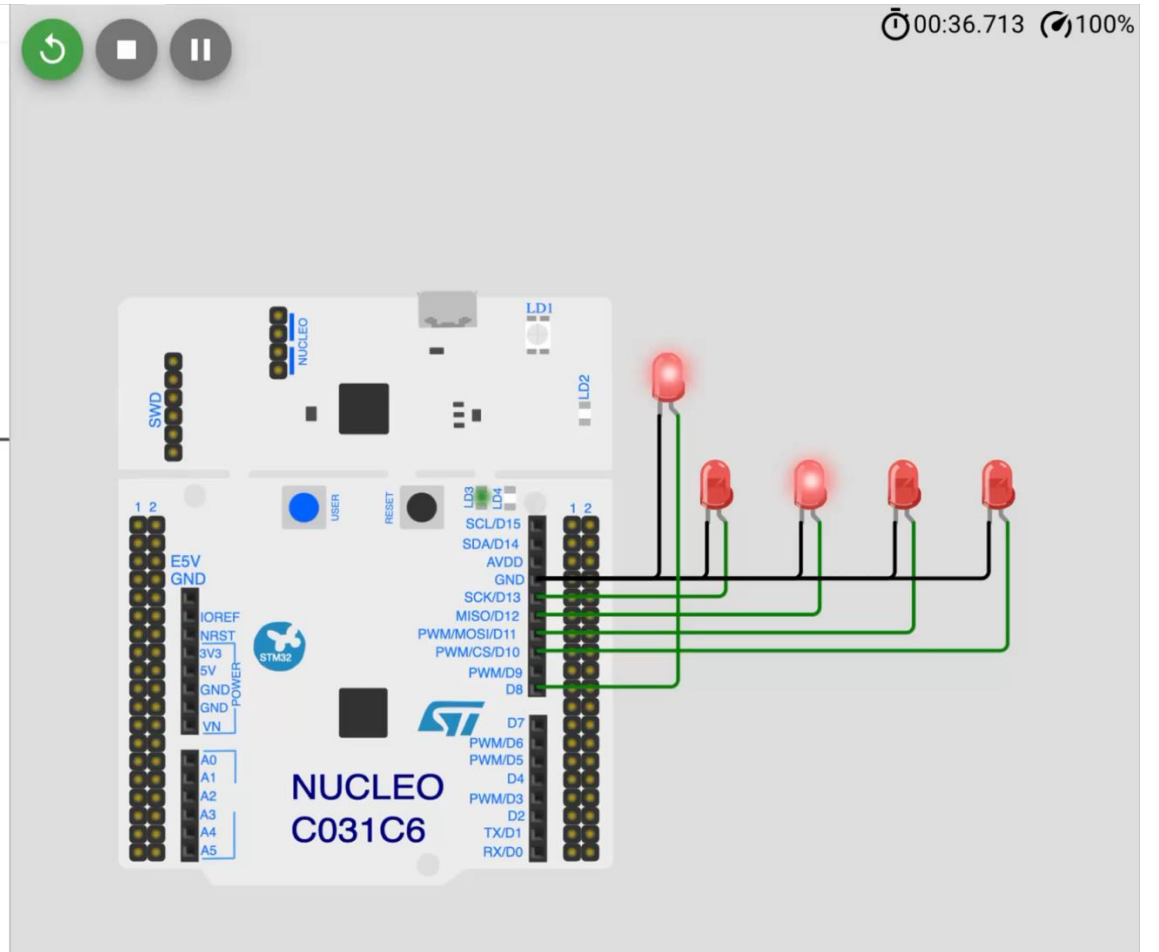
Virtual lab session: Running LEDs

- Can you light up one more LED along with the four running LEDs?

Virtual lab session: Running LEDs

- Can you light up one more LED along with the four running LEDs?

```
1 // 4-bit binary counter using LEDs (each LED toggles at different rates)
2
3 int ledPin[] = {13, 12, 11, 10};           // LED pins: MSB → LSB
4 int ledDelay[] = {8000, 4000, 2000, 1000}; // toggle interval (ms)
5 unsigned long lastChangeTime[4];           // store last toggle time of each LED
6 int ledState[4] = {LOW, LOW, LOW, LOW};    // store current state (LOW/HIGH)
7 unsigned long currTime;                    // current time variable
8
9 void setup() {
10     currTime = 0;                          // initialize timer
11     for (int i = 0; i < 4; i++) {
12         pinMode(ledPin[i], OUTPUT);         // set pin as output
13         digitalWrite(ledPin[i], ledState[i]); // turn LED off initially
14         lastChangeTime[i] = millis();       // record initial time
15     }
16     pinMode(8, OUTPUT);
17     digitalWrite(8, LOW);
18 }
19
20 void loop() {
21     currTime = millis();                    // get current time
22
23     for (int i = 0; i < 4; i++) {           // check all LEDs
24         if (currTime - lastChangeTime[i] >= ledDelay[i]) {
25             // time to toggle the LED
26             ledState[i] = !ledState[i];     // flip LED state
27             digitalWrite(ledPin[i], ledState[i]); // apply new state
28             lastChangeTime[i] = currTime;    // reset timer for that LED
29         }
30
31         digitalWrite(8, HIGH);
32     }
33 }
```



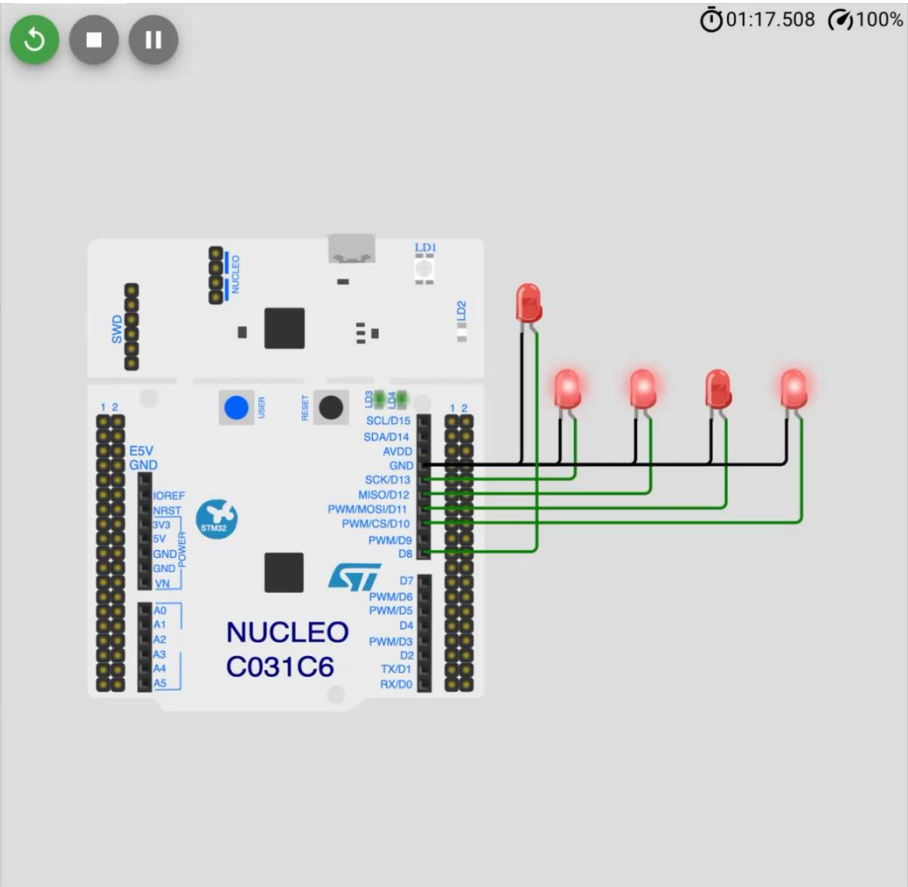
Virtual lab session: Running LEDs

- Can you light up one more LED only when the four running LEDs are on?

Virtual lab session: Running LEDs

- Can you light up one more LED only when the four running LEDs are on?

```
1 // 4-bit binary counter using LEDs (each LED toggles at different rates)
2
3 int ledPin[] = {13, 12, 11, 10}; // LED pins: MSB → LSB
4 int ledDelay[] = {8000, 4000, 2000, 1000}; // toggle interval (ms)
5 unsigned long lastChangeTime[4]; // store last toggle time of each LED
6 int ledState[4] = {LOW, LOW, LOW, LOW}; // store current state (LOW/HIGH)
7 unsigned long currTime; // current time variable
8
9 void setup() {
10     currTime = 0; // initialize timer
11     for (int i = 0; i < 4; i++) {
12         pinMode(ledPin[i], OUTPUT); // set pin as output
13         digitalWrite(ledPin[i], ledState[i]); // turn LED off initially
14         lastChangeTime[i] = millis(); // record initial time
15     }
16     pinMode(8, OUTPUT);
17     digitalWrite(8, LOW);
18 }
19
20 void loop() {
21     currTime = millis(); // get current time
22
23     for (int i = 0; i < 4; i++) {
24         // check all LEDs
25         if (currTime - lastChangeTime[i] >= ledDelay[i]) {
26             // time to toggle the LED
27             ledState[i] = !ledState[i]; // flip LED state
28             digitalWrite(ledPin[i], ledState[i]); // apply new state
29             lastChangeTime[i] = currTime; // reset timer for that LED
30         }
31
32         if (ledState[0] == HIGH && ledState[1] == HIGH &&
33             ledState[2] == HIGH && ledState[3] == HIGH) {
34             digitalWrite(8, HIGH);
35         } else {
36             digitalWrite(8, LOW);
37         }
38     }
39 }
```



Course overview

	Monday	Tuesday	Wednesday	Thursday	Friday		Week	Topic of Each Week	Homework
Jan	5	6	7	8	9		1	Introduction to embedded systems and IoT	
	12	13	14	15	16		2	Electronics and digital logic	
	19	20	21	22	23		3	C programming basics	
	26	27	28	29	30		4	Embedded software organization	
Feb	2	3	4	5	6		5	Design and development process	Feb 4: HW1
	9	10	11	12	13				
	16	17	18	19	20				
	23	24	25	26	27				
Mar	2	3	4	5	6		6	I/O ports, switch and LED interfacing	
	9	10	11	12	13		7	Digital sampling, ADC and DAC	
	16	17	18	19	20		8	Memory and processor bus, timer and counters	
	23	24	25	26	27		9	Timing constraints and real-time CPU sched	Mar 25: HW2
Apr	30	31	1	2	3		10	Internet of things basics	
	6	7	8	9	10		11	Bluetooth and CAN	
	13	14	15	16	17		12	Machine learning introduction	
	20	21	22	23	24		13	Embedded machine learning applications	Apr 22: HW3
	27	28	29	30	1		14	Review for final exam and guest lectures	
May	4	5	6	7	8				May 9: Final Exam 1pm to 3pm

Recap: von Neumann architecture

- Instructions are fetched from ROM on the same bus as data fetched from RAM
- Software is an ordered sequence of very specific instructions
- The processor executes the software by retrieving and interpreting these instructions one at a time

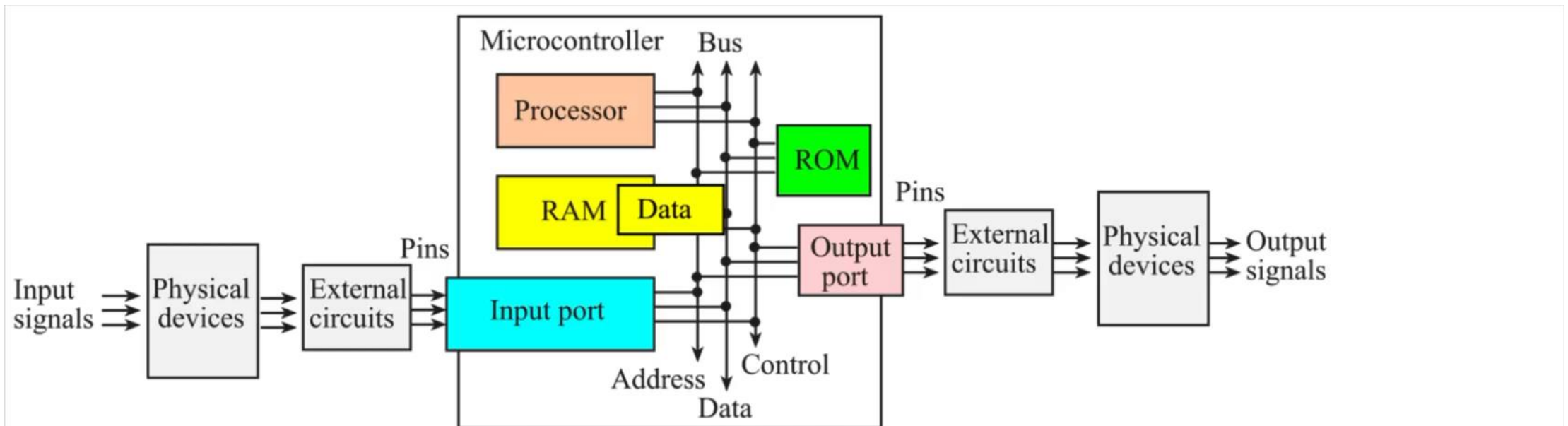


Figure 1.1.1. The basic components of a von Neumann computer including processor, memory and I/O connected with by a single bus.

Recap: Harvard architecture

- The processor can simultaneously fetch an instruction out of flash ROM using the ICode bus while it writes data into RAM using the system bus

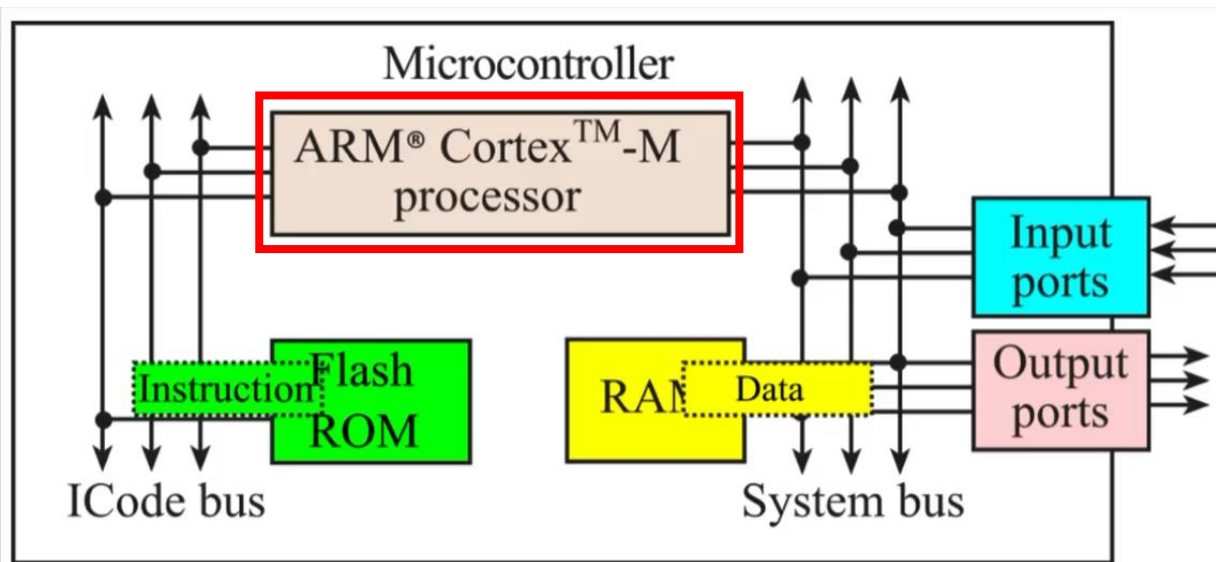
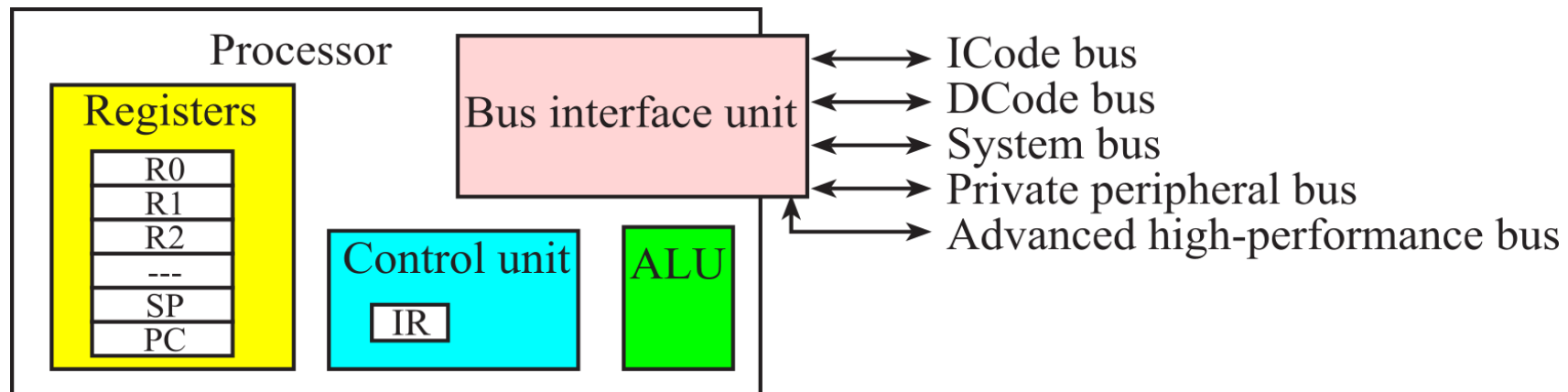


Figure 1.1.2. Harvard architecture of an ARM Cortex-M4. The three signals on each bus are address, data, control. If all these components exist on a single chip, it is called a microcontroller.

Four basic components of ARM Cortex-M processor

- Control unit (CU): Operating – read or write
- Bus interface units (BIU): Interacting across modules
- Arithmetic logic unit (ALU): Arithmetic and logic operations
- CPU Registers: Holding data, addresses, and intermediate results



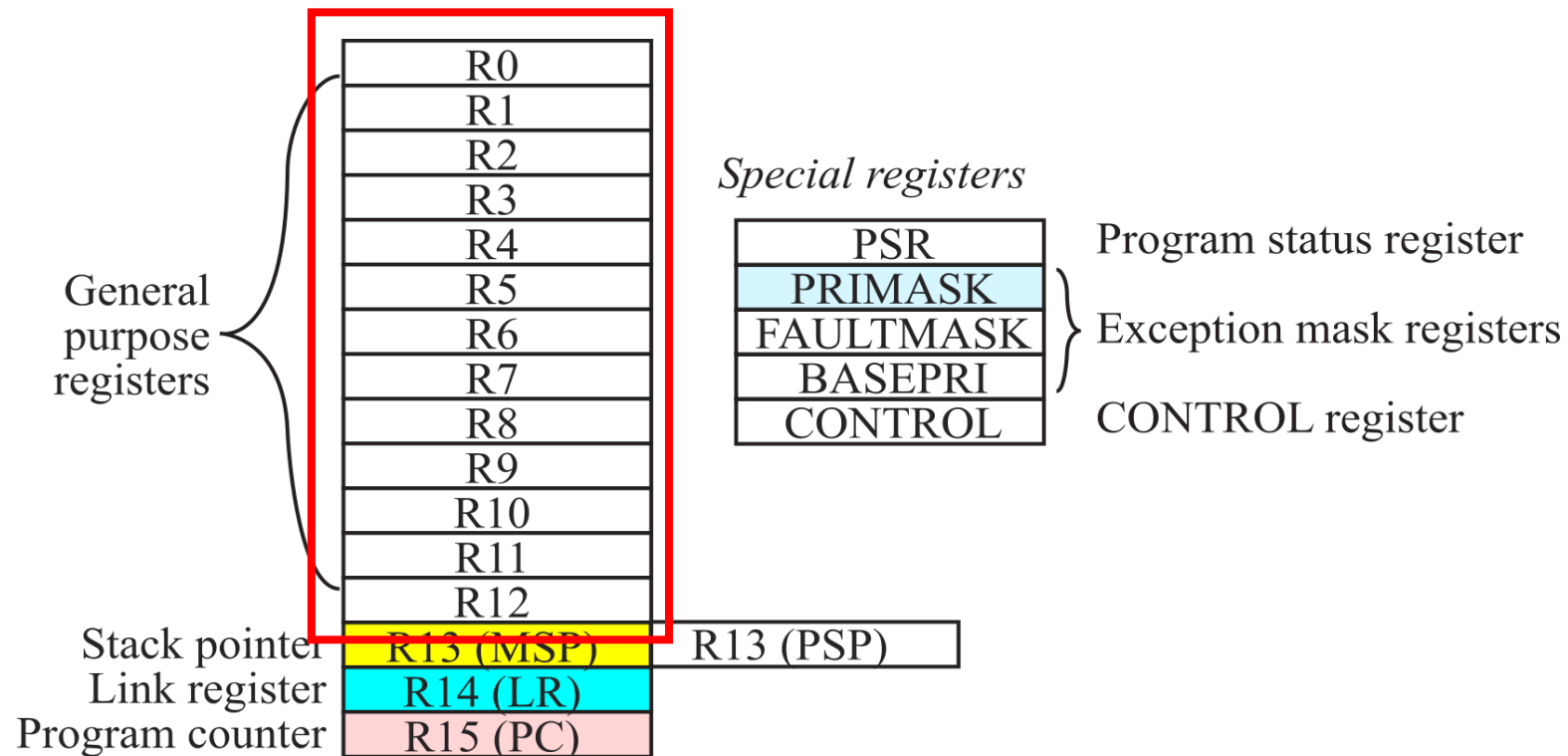
Four phases of execution

<i>Phase</i>	<i>Function</i>	<i>Bus</i>	<i>Address</i>	<i>Comment</i>
1	Instruction fetch	Read	PC++	Put into IR
2	Data read	Read	EAR	Data passes through ALU
3	Operation	-	-	ALU operations, set PSR
4	Data store	Write	EAR	Results stored in memory

- The effective address register (EAR) contains the memory address used to fetch the data needed for the current instruction

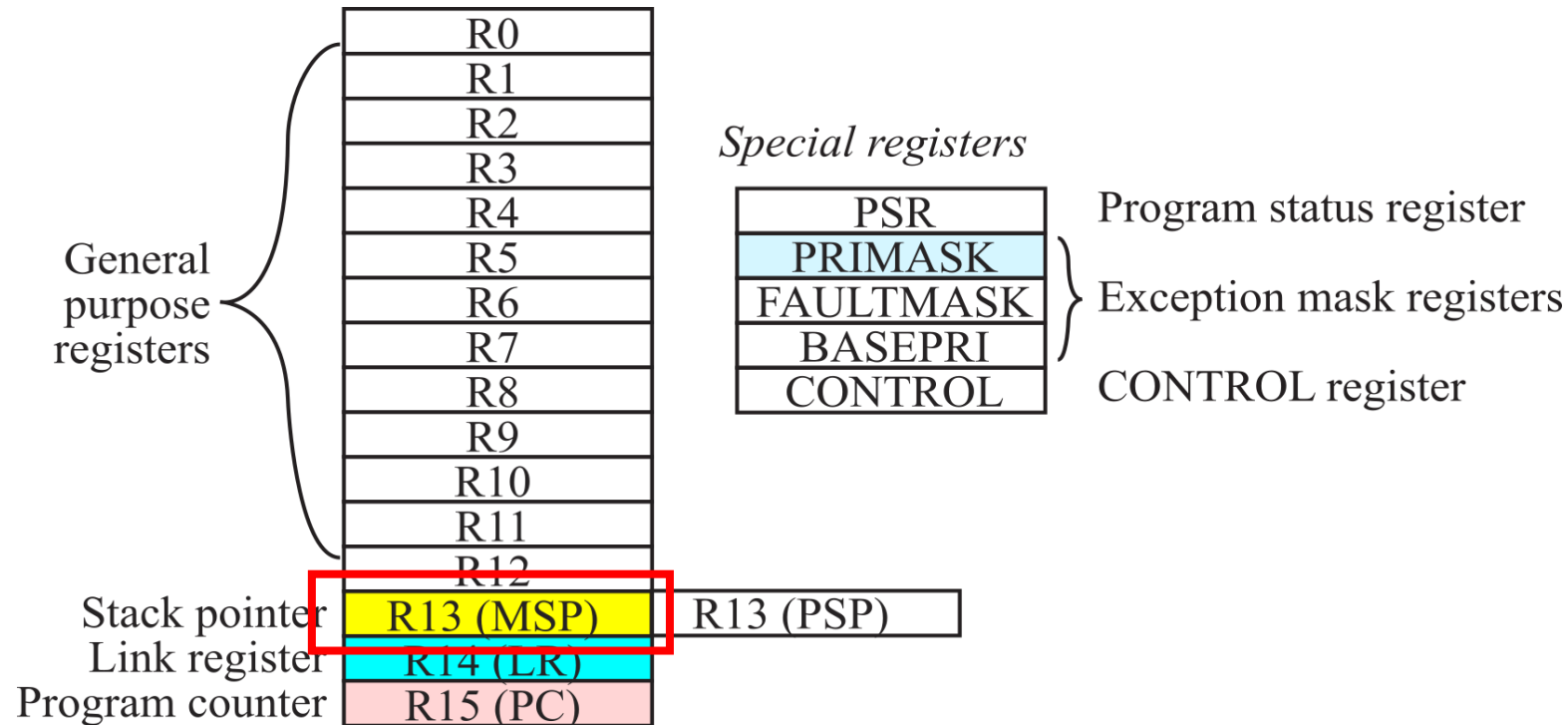
CPU registers

- General purpose registers and contain either data or addresses
 - **Example:** ADD R1, R2, R3 (This means $R1 = R2 + R3$). The values in registers R2 and R3 are fed into the ALU, and the result is placed back into register R1



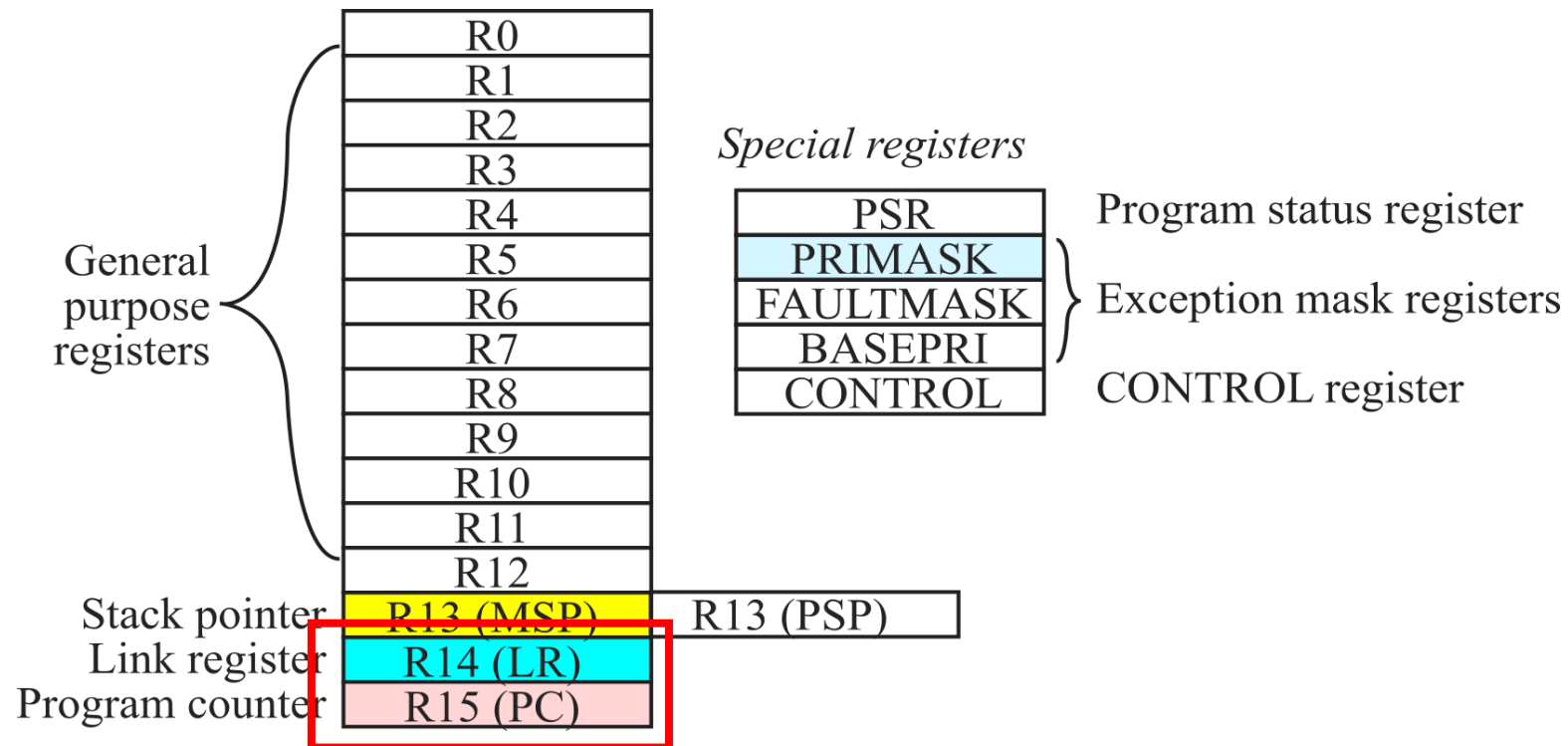
CPU registers

- Stack pointer (SP) points to the top element of the stack
 - Stack is a special region of memory that operates on Last-In, First-Out principle
 - SP is a special CPU register that always points to the top of the stack

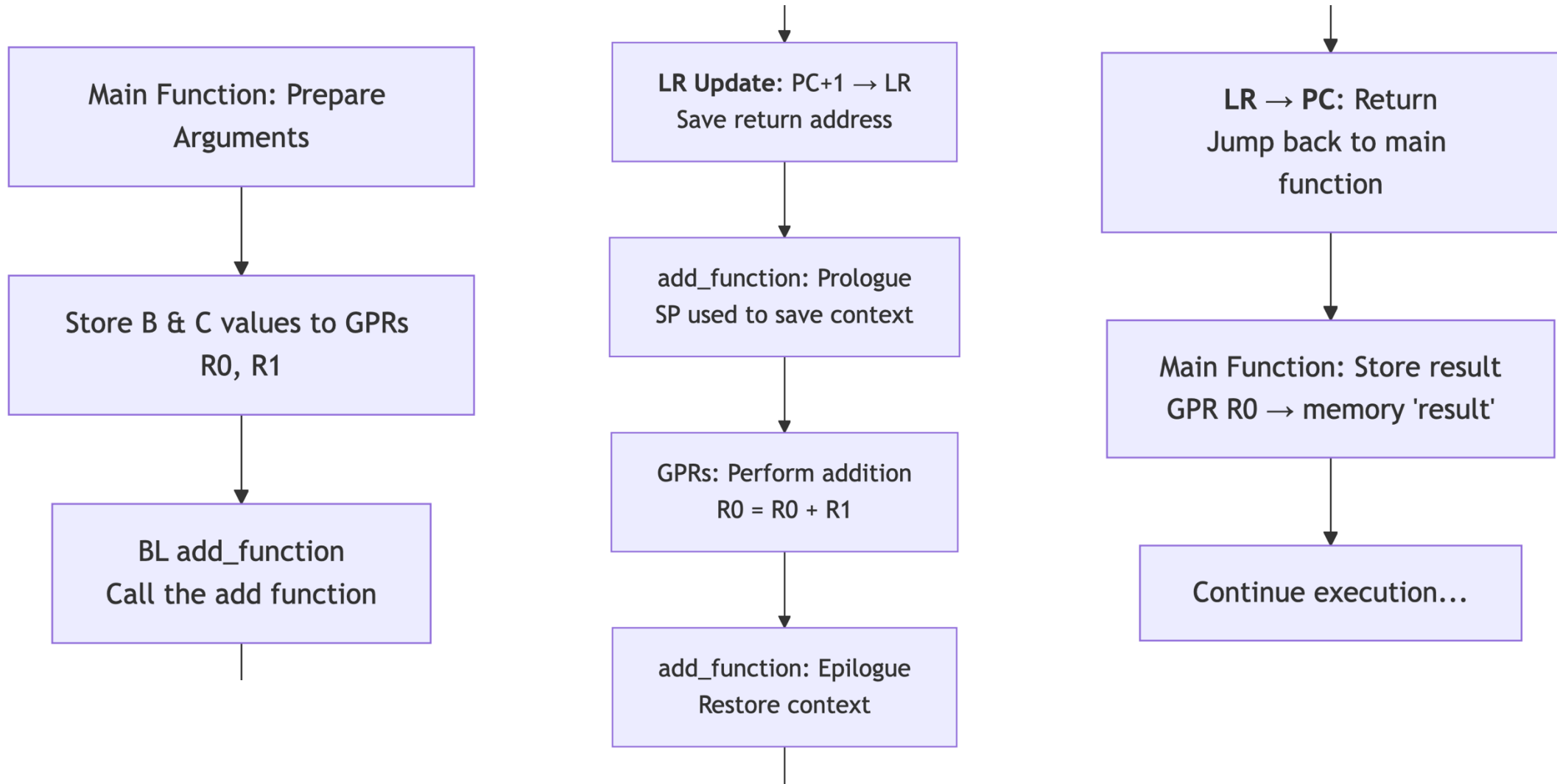


CPU registers

- Link register (LR) is used to store the return location for functions
- Program counter (PC) points to the next instruction to be fetched from memory

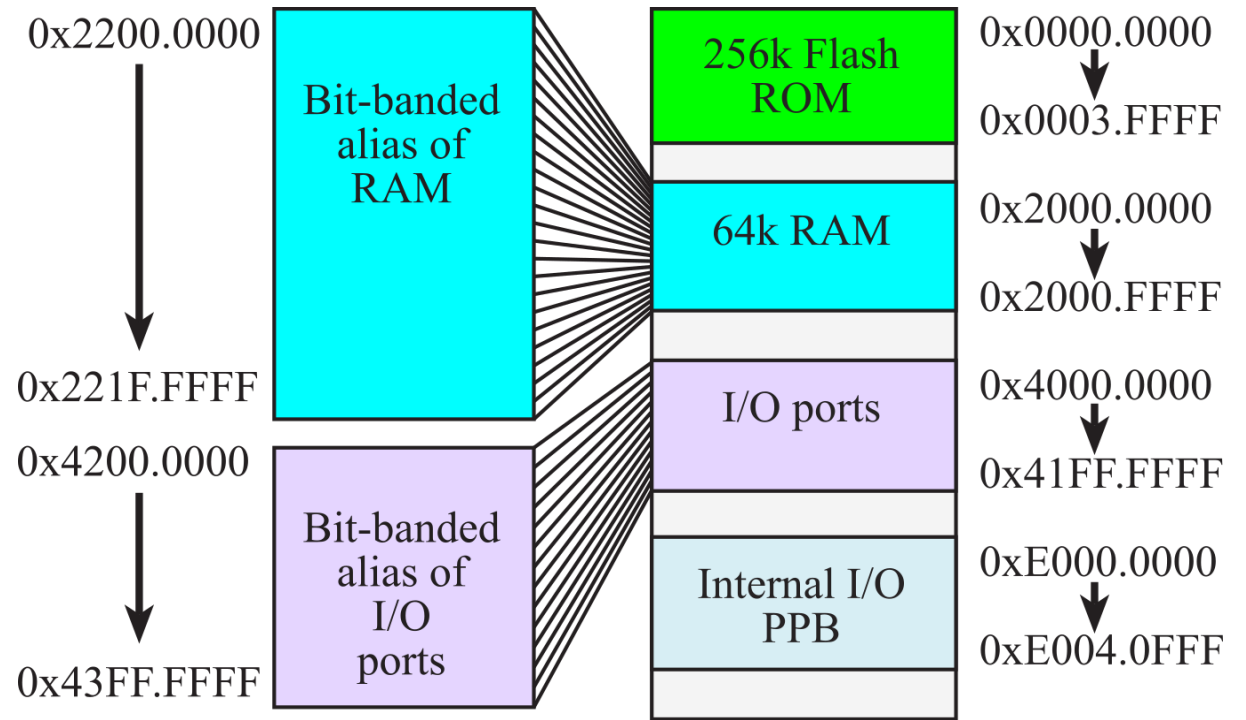


An example: $\text{result} = \text{add}(\text{B}, \text{C})$



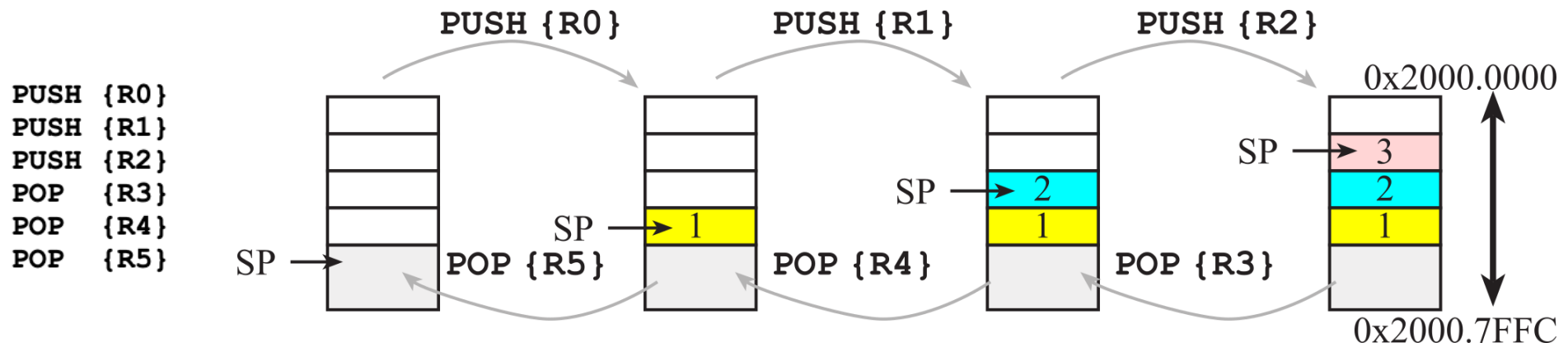
Memory map and bit banding

- Memory map: The address of the memory
- Flash ROM begins at address 0x00000000, RAM begins at 0x20000000
- Bit-banding to allow read/write access to individual bits in RAM and some bits in the I/O space



Stack

- The stack is a last-in-first-out temporary storage
- The SP points to the 32-bit data on the top of the stack
- The stack grows downwards in memory as we push data on to it so
- We refer to the most recent item as the "top of the stack" it is actually the item stored at the lowest address



Reset

- A reset occurs immediately after power is applied and can also occur by pushing the reset button available on most boards
- The 32-bit value at flash ROM location 0 is loaded into the SP. All stack accesses are word aligned
- A reset also loads the 32-bit value at location 4 into the PC. This value is called the reset vector. All instructions are halfword-aligned
- On reset, the processor initializes the LR to 0xFFFFFFFF.

What is IoT?

- ✓ Internet of Things is the **network of smart physical objects**
 - physical objects (e.g. devices, vehicles, buildings, etc.) **embedded with** sensors, computation unit, memory unit, power source, and network connectivity,
 - which enables the physical object to **collect and exchange data**,
 - **analyze** the collected data to extract new insight and **respond** accordingly.
- ✓ Goal of IoT is to “**connect the unconnected**”
 - “Things” or “objects” **that were not supposed to be connected** to the Internet



- IoT did the **technology transition** in traditional computer networks

Slide credit:



Dr. Manas Khatua

Associate Professor
Computer Science and Engineering
Indian Institute of Technology Guwahati

Guwahati, Assam, India - 781039.

✉ manaskhatua[at]iitg[dot]ac[dot]in
manaskhatua[at]gmail[dot]com

What is IoT?

- IoT is the Unifications of technologies:
 - ✓ Embedded Systems
 - ✓ Low Power and Low Rate Network
 - ✓ Internet
 - ✓ Cloud Computing
 - ✓ Data Analytics
 - ✓ Bigdata
 - ✓ Edge Intelligence
 - ✓ Network Security and Data Security
 - ✓ Software Defined Networks
 - ✓ Etc.
- Alternate Definition:

“The Internet of Things (IoT) is the **network of physical objects** that contain embedded technology to **communicate** and **sense** or **interact** with their **internal states** or the **external environment**.” – Gartner Research*

Brief History of Initial Phase of IoT

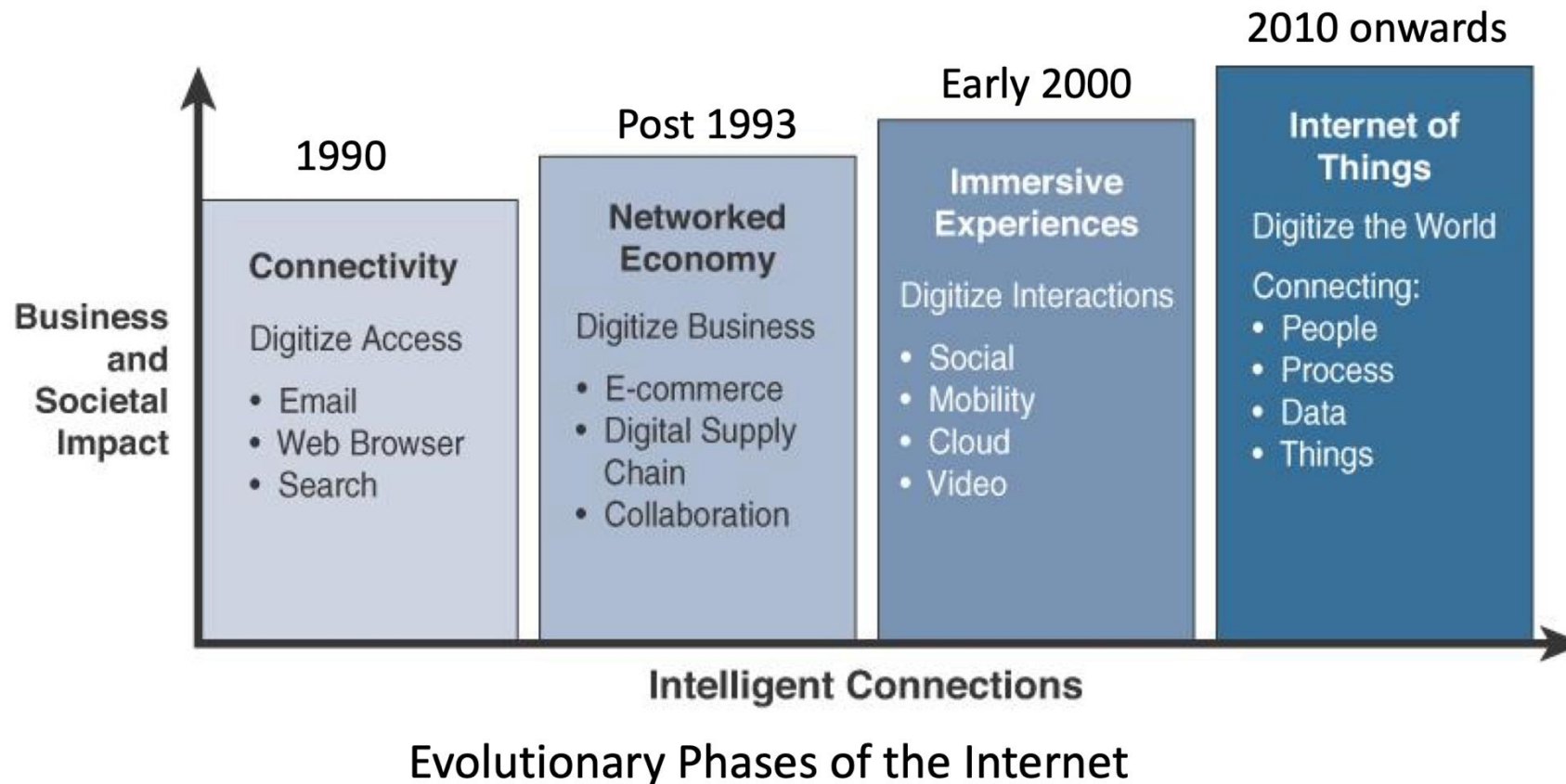
- The term "**Internet of Things**" was likely coined by **Kevin Ashton** of Procter & Gamble, later MIT's Auto-ID Center, in 1999.
 - Kevin Ashton said –
“In 20th century, **computers were brains without senses** — they only knew what we told them.”
“Now in 21st century, **computers are sensing things for themselves!**”
- Early 1980s at the Carnegie Mellon University, a group of students created a way to get their campus **Coca-Cola vending machine** to report on its contents **through a network** in order to save them the trek if the machine was out of Coke.

Brief History of Initial Phase of IoT

- In 1990, John Romkey, developer of the first TCP/IP stack for IBM PC in 1983, connected a toaster to the internet for the first time.
- In 1991, a group of students at the University of Cambridge used a web camera to report on coffee available in their computer labs coffee pot.
- At the beginning of the 21st Century, LG Electronics introduced the world's first refrigerator connected to the internet

Brief History of Initial Phase of IoT

- The popularity of IoT did not accelerate until 2010/2011 and reached mass market from 2013-14.
- Definition of the IoT has also evolved over time.



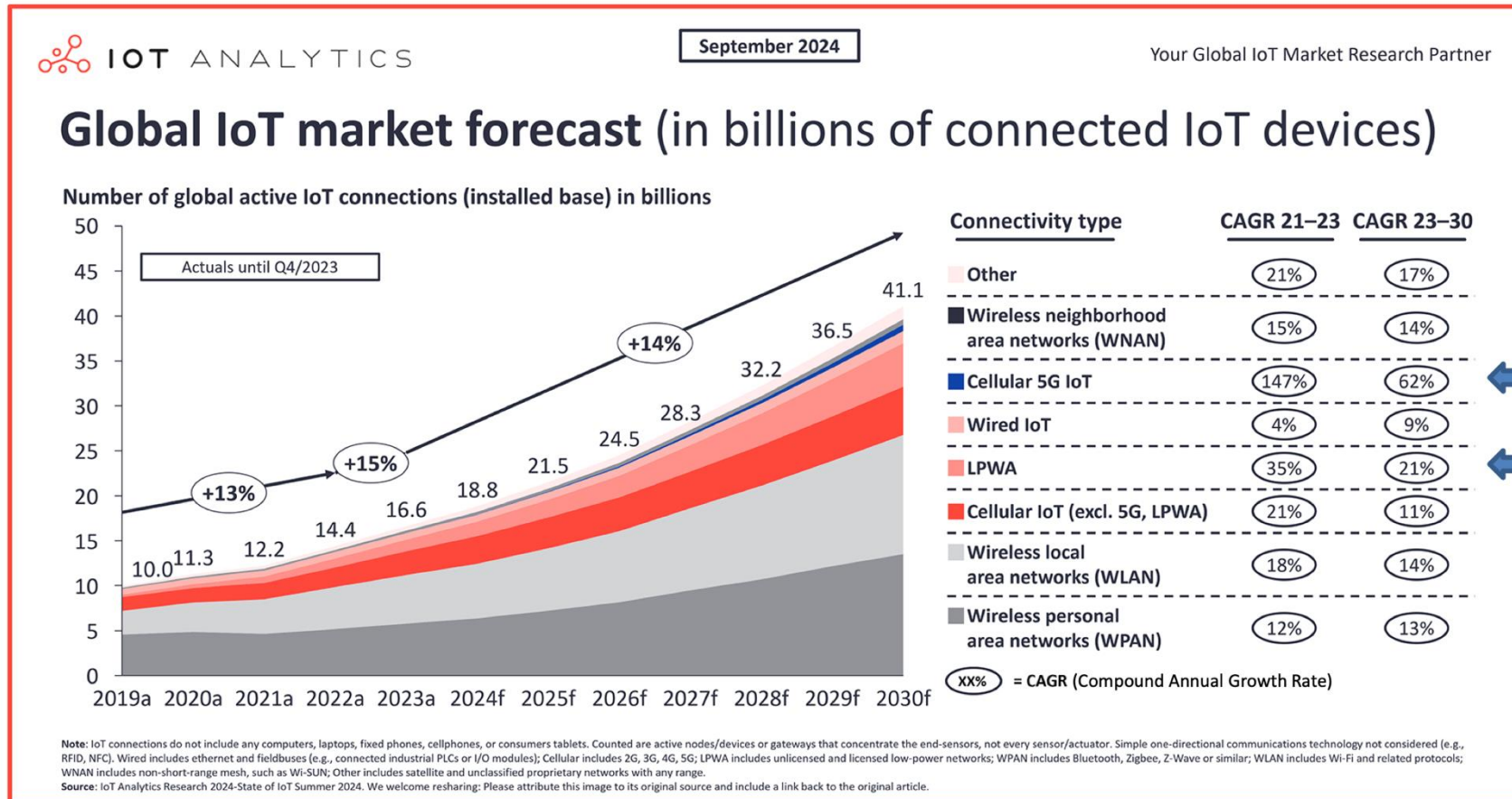
Benefits of IoT

- **Real-Time Monitoring**
 - IoT sensors can **monitor equipment and processes** in real-time, providing valuable sensed data that can be used to streamline operations, reduce waste, and increase output.
- **Automation of Processes**
 - **Machines can assemble parts** with more precision and speed, resulting in fewer errors during assembly
 - **Robots can** very rapidly **detect faults** that may not be detected by the human eye
- **Improved Efficiency or Productivity**
 - IoT helps to **enhance productivity by** streamlining processes, automating tasks, and providing real-time data insights.
- **Predictive Maintenance**
 - Continuous monitoring of systems and processes to **identify key indicators of problems** before they result in downtime or system failure

Benefits of IoT

- **Improved or New Insights**
 - IoT generates vast amounts of data that **can be analyzed to identify trends, inefficiencies, and areas for improvement**. Organizations can leverage these insights to enhance productivity and operational effectiveness.
- **Cost Reduction**
 - When an **organization can improve** system uptime, automate processes, reduce the risk of failure and gain insights that support better decision making, and reduce resource usage, the result is efficiency and cost savings
- **Optimized Work Environments**
 - IoT technology can help **create more comfortable and efficient workspaces**
- **Adaptability**
 - The **ability to adapt** to new business requirements, customer needs, and changing conditions, or scale the deployment in response to business growth or customer requirements

Growth of IoT Devices



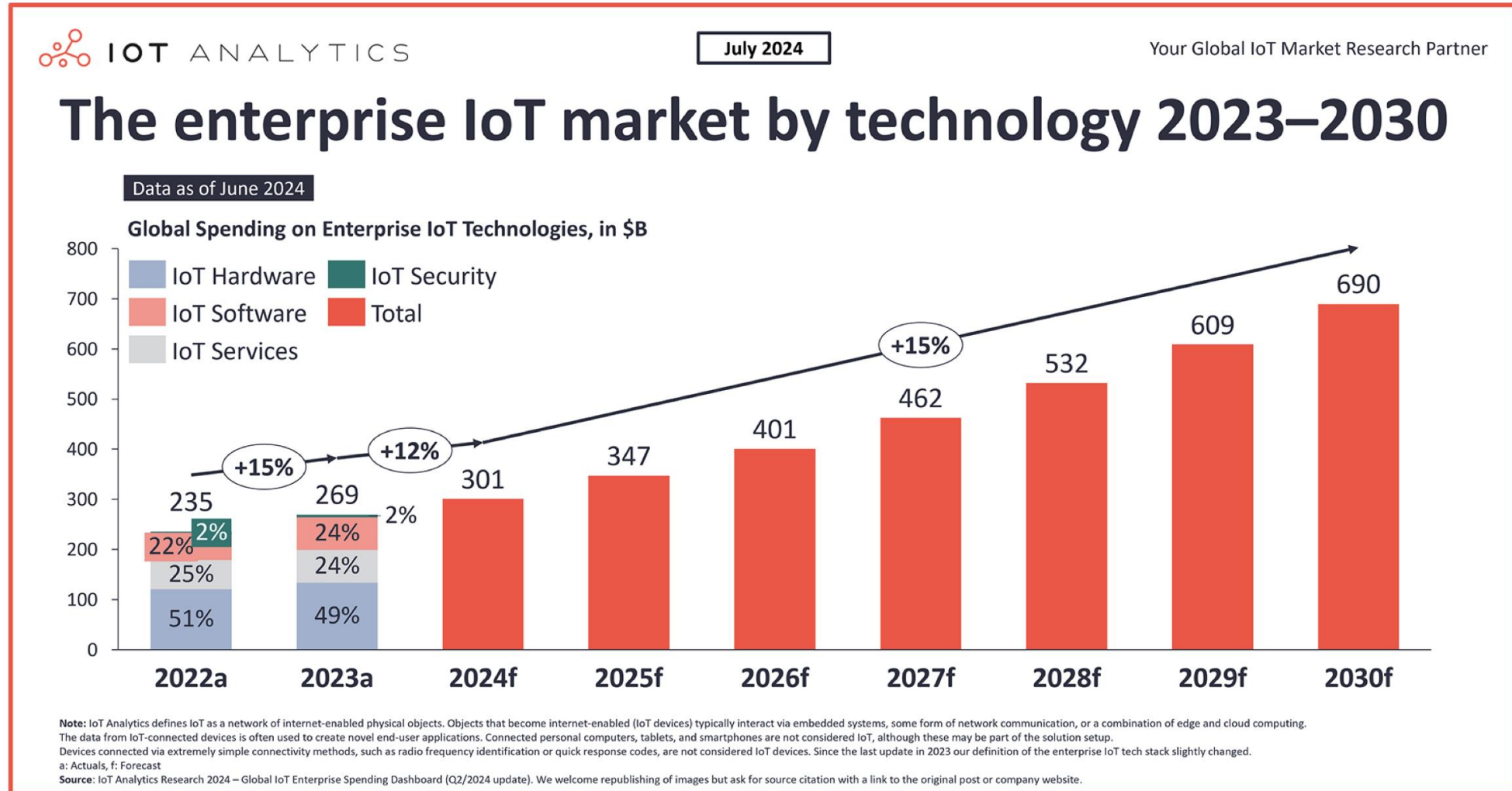
IoT Analytics' prediction

IoT devices **excluding** PC, Laptop, Smartphone, Tablet, and one directional technology like RFID, NFC

Image Source: <https://iot-analytics.com/number-connected-iot-devices/>

Wired: Eth, SCADA network (Modbus, Fieldbus, etc.)
Cellular: 2G, 3G, 4G, 5G; **LPWA:** LoRa, HaLow
WPAN: BLE, ZigBee, Z-Wave, etc.; **WLAN:** WiFi
WNAN: Non-short-range mesh e.g. Wi-SUN

Global Spending on IoT



Where is IoT?

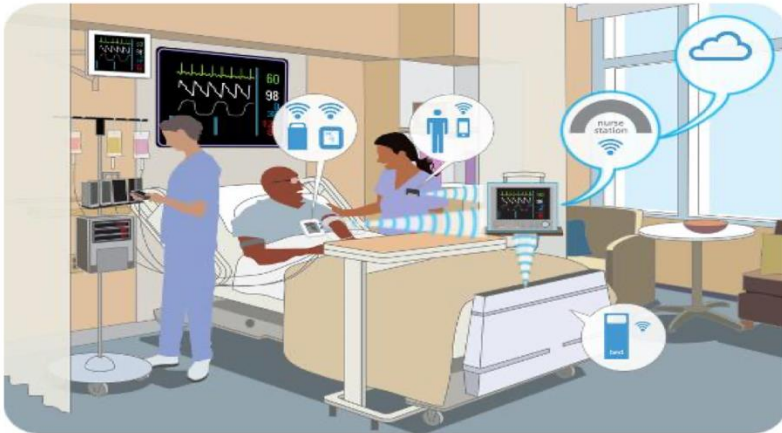


Wearable
Tech Devices



Smart Appliances

It's everywhere!

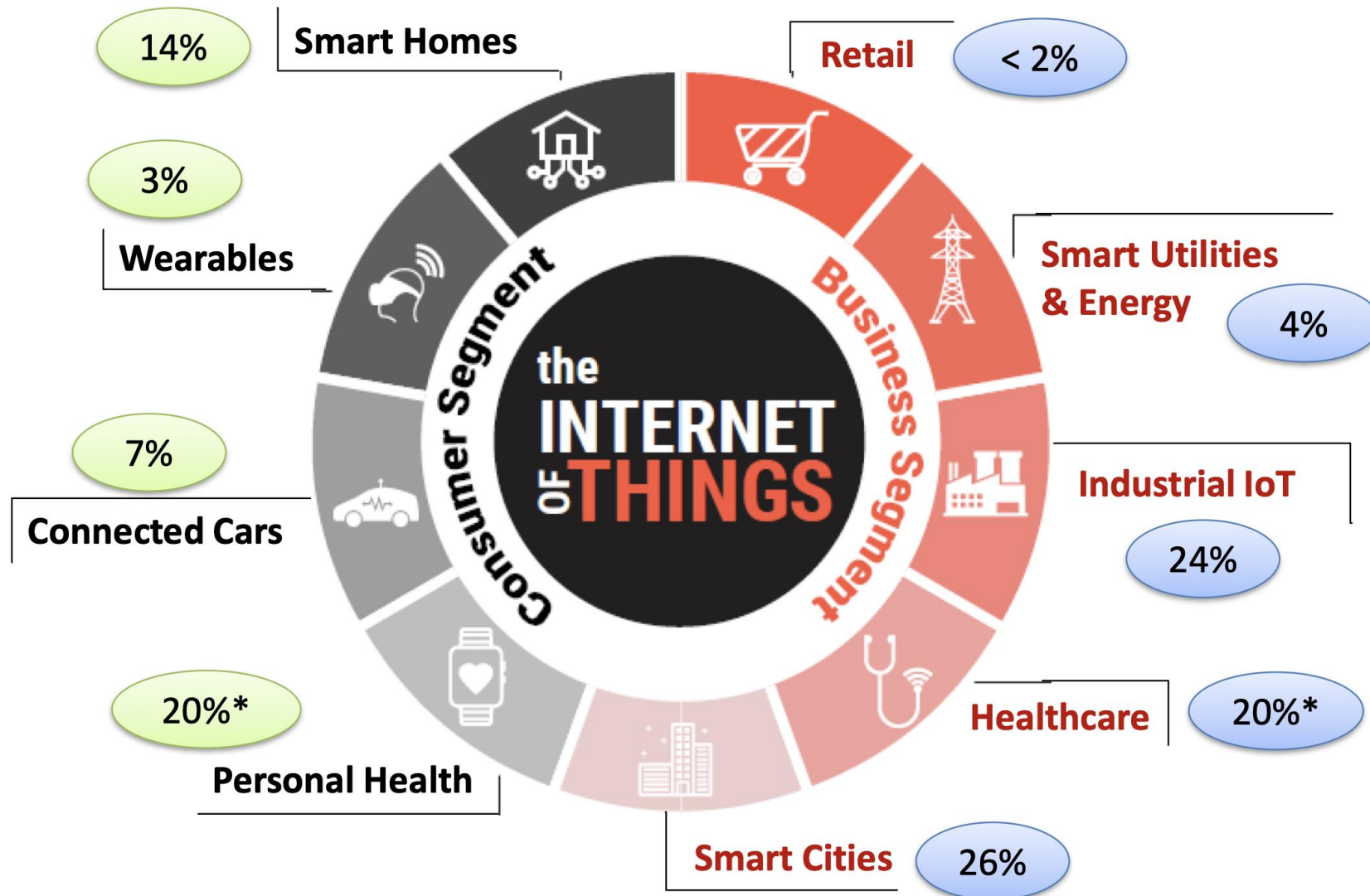


Healthcare

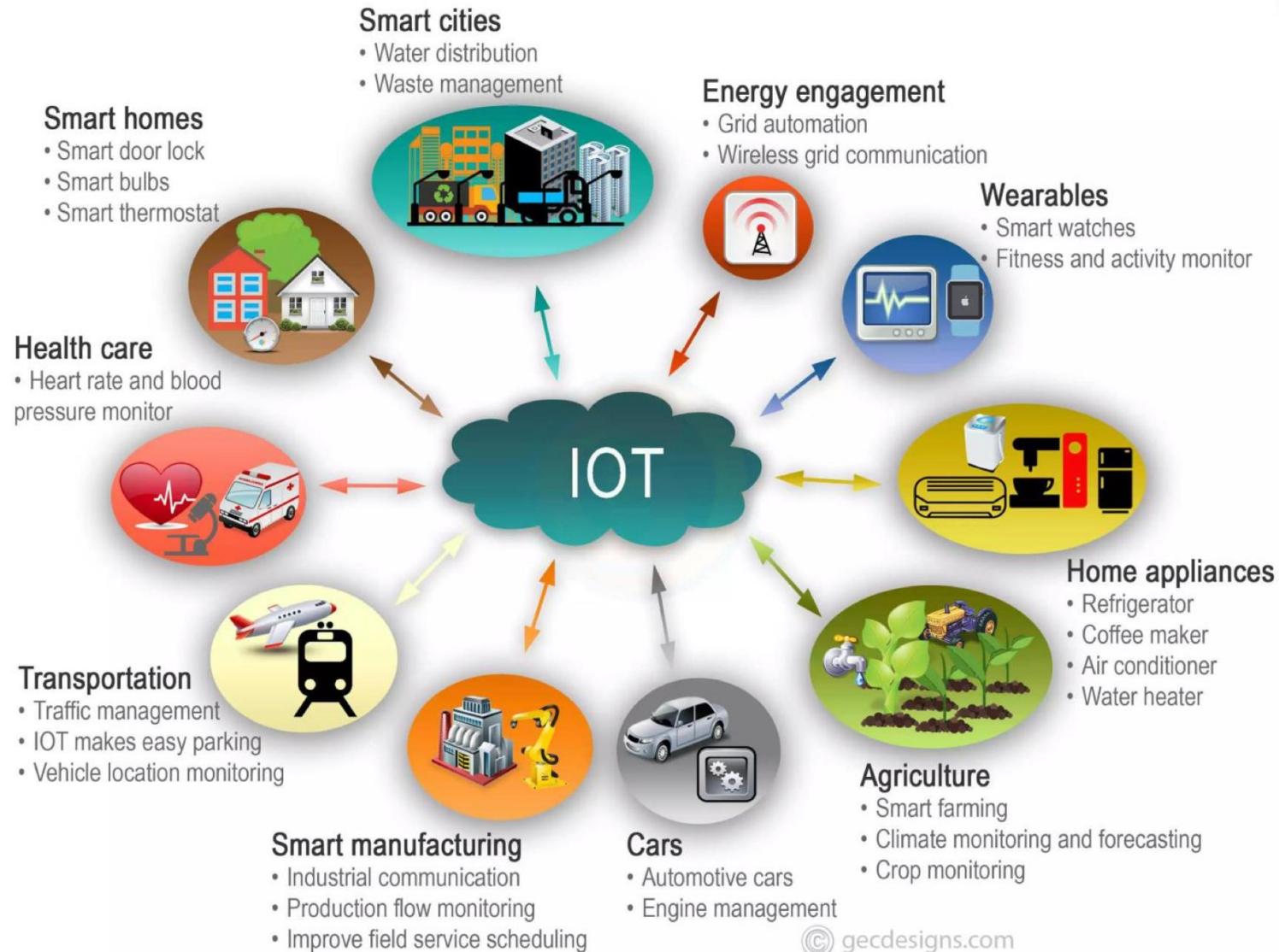


Industry Automation
and Monitoring

Global IoT Market Share



Applications of IoT



Main Challenges in IoT Implementation

Sensors

- limited resources
- limited types of sensors

Scale

- millions of devices are connected to form IoT

Low Power Network

- devices should remain connected to the network for years
- high network latency
- can't use traditional communication protocols

Interoperability

- various protocol, various architecture
- unavailability of standardized platform
- different technology leads to interoperability issue

Bigdata & Data analytics

- massive amount of sensor data
- different sources and various forms
- extract intelligence from the heaps of data

Privacy

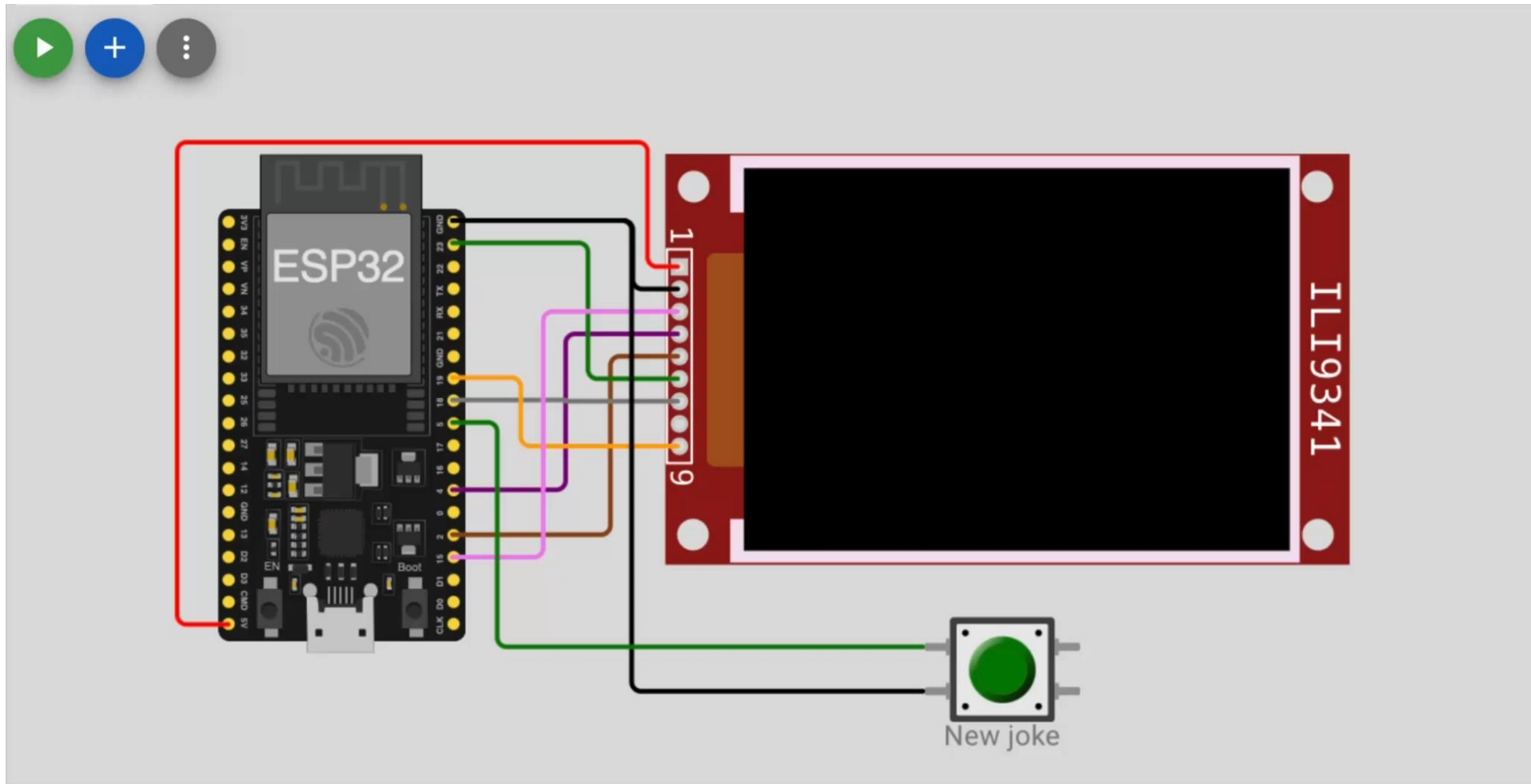
- which personal data to share with whom
- how to control

Security

- “things” becomes connected, so security becomes complex

Virtual lab session: An example of joke machine

- <https://wokwi.com/projects/342032431249883731>



The joke is grabbed from the internet

- <https://v2.jokeapi.dev/joke/Programming>

```
{
  "error": false,
  "category": "Programming",
  "type": "twopart",
  "setup": "Hey baby I wish your name was asynchronous...",
  "delivery": "... so you'd give me a callback.",
  "flags": {
    "nsfw": false,
    "religious": false,
    "political": false,
    "racist": false,
    "sexist": false,
    "explicit": false
  },
  "id": 53,
  "safe": true,
  "lang": "en"
}
```

The joke is grabbed from the internet

```
1  /*
2   ESP32 HTTPClient Jokes API Example
3
4   https://wokwi.com/projects/342032431249883731
5
6   Copyright (C) 2022, Uri Shaked
7  */
8
9  #include <WiFi.h>
10 #include <HTTPClient.h>
11 #include <ArduinoJson.h>
12 #include <Adafruit_GFX.h>
13 #include <Adafruit_ILI9341.h>
14
15 const char* ssid = "Wokwi-GUEST";
16 const char* password = "";
17
18 #define BTN_PIN 5
19 #define TFT_DC 2
20 #define TFT_CS 15
21 Adafruit_ILI9341 tft = Adafruit_ILI9341(TFT_CS, TFT_DC);
22
23 const String url = "https://v2.jokeapi.dev/joke/Programming";
24
25 String getJoke() {
26   HTTPClient http;
27   http.useHTTP10(true);
28   http.begin(url);
29   http.GET();
30   String result = http.getString();
31
32   DynamicJsonDocument doc(2048);
33   DeserializationError error = deserializeJson(doc, result);
34
35   // Test if parsing succeeds.
36   if (error) {
37     Serial.print("deserializeJson() failed: ");
38     Serial.println(error.c_str());
39     return "<error>";
40   }
```

```
42   String type = doc["type"].as<String>();
43   String joke = doc["joke"].as<String>();
44   String setup = doc["setup"].as<String>();
45   String delivery = doc["delivery"].as<String>();
46   http.end();
47   return type.equals("single") ? joke : setup + " " + delivery;
48 }
49
50 void nextJoke() {
51   tft.setTextColor(ILI9341_WHITE);
52   tft.println("\nLoading joke...");
53
54   String joke = getJoke();
55   tft.setTextColor(ILI9341_GREEN);
56   tft.println(joke);
57 }
58
59 void setup() {
60   pinMode(BTN_PIN, INPUT_PULLUP);
61
62   WiFi.begin(ssid, password, 6);
63
64   tft.begin();
65   tft.setRotation(1);
66
67   tft.setTextColor(ILI9341_WHITE);
68   tft.setTextSize(2);
69   tft.print("Connecting to WiFi");
70
71   while (WiFi.status() != WL_CONNECTED) {
72     delay(100);
73     tft.print(".");
74   }
75
76   tft.print("\nOK! IP=");
77   tft.println(WiFi.localIP());
78
79   nextJoke();
80 }
```

```
82 void loop() {
83   if (digitalRead(BTN_PIN) == LOW) {
84     tft.fillScreen(ILI9341_BLACK);
85     tft.setCursor(0, 0);
86     nextJoke();
87   }
88
89   delay(100);
90 }
```

Thank you!